uc3m | Universidad Carlos III de Madrid

Master Degree in
Applied and Computational Mathematics
2025-2026

Master Thesis

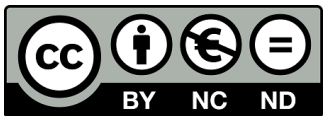
Physics-Informed and Data-Driven Neural Networks for Option Pricing, Calibration, and Greeks Estimation under the Heston Model

---

Jose Enrique Zafra Mena

Jorge Morón Vidal
Francisco Manuel Bernal Martínez
Madrid, 2026

AVOID PLAGIARISM
The University uses the **Turnitin Feedback Studio** for the delivery of student work. This program compares the originality of the work delivered by each student with millions of electronic resources and detects those parts of the text that are copied and pasted. Plagiarizing in a TFM is considered a **Serious Misconduct**, and may result in permanent expulsion from the University.



This work is licensed under Creative Commons **Attribution – Non Commercial – Non Derivatives**

SUMMARY

Neural methods can accelerate derivative pricing, but it remains unclear whether this speed can be used inside a complete Heston workflow without compromising calibration reliability and Greek accuracy. Existing studies often evaluate neural pricers mainly through pricing errors or treat calibration and sensitivity computation as separate tasks. This leaves open the question of how neural components behave when they are connected in the same benchmarked pipeline and judged not only by average accuracy, but also by downstream calibration performance, spatial error structure, and derivative quality.

This thesis studies that question for European vanilla put options under the Heston stochastic-volatility model. A deterministic Heston-COS pricing engine with Brent implied-volatility inversion is used to generate controlled synthetic references, while semi-analytical Heston Greeks provide benchmarks for sensitivity diagnostics. On top of this reference layer, the thesis develops a supervised ANN implied-volatility surrogate, uses it inside a CaNN calibration workflow, and evaluates a physics-informed neural network as a differentiable pricing surface. Sobolev training and an analytic control variate are then tested as two strategies for improving Greek accuracy in difficult short-maturity at-the-money regions.

The results show that the ANN/CaNN block gives the strongest calibration performance, with very small quote-level residuals and improved Heston parameter recovery in the synthetic benchmark. The PINN provides a fast differentiable pricing engine, but its Greek accuracy is more fragile than its price accuracy. Sobolev augmentation reduces global Greek errors, while the analytic control variate gives the strongest hard-region Delta and Gamma improvements. Overall, the thesis shows that neural methods are useful for Heston derivative analytics when they are embedded in a disciplined numerical benchmark that evaluates pricing, calibration, and sensitivities separately.

Acortar o incluso adaptar si hay cambios

Keywords: Heston model; derivative pricing; implied volatility; COS method; neural surrogate; calibration; physics-informed neural networks; automatic differentiation; Greeks; Sobolev training; control variates.

DEDICATION

CONTENTS

LIST OF FIGURES

LIST OF TABLES

1. INTRODUCTION

1.1. Motivation

Financial derivatives are financial contracts whose value is linked to the behaviour of an underlying market variable, such as an asset price, an interest rate, an index level, or an exchange rate [1]. The payoff of the contract is then determined by how that underlying variable evolves up to the relevant exercise or maturity date. Options are one of the most common examples: they give the holder the right, but not the obligation, to buy or sell the underlying asset at a fixed strike price K, with the contract ending at a specified maturity date T. A European option can only be exercised at that maturity date, which makes it simple to study in a controlled way but still rich enough to show the main numerical difficulties of derivative pricing.

At a high level, derivative pricing is inherently a stochastic problem because the underlying assets are modelled as stochastic processes. The uncertainty in future asset prices is therefore built directly into the mathematical framework used for valuation. While the formal machinery of stochastic calculus is introduced in Chapter 2, the key point for now is that randomness is an intrinsic feature of the pricing model rather than an external source of noise.

In practice, option pricing is not only a question of computing one price. The same pricing model is often used many times: to generate prices for different strikes and maturities, to fit model parameters to quoted option prices, and to measure how sensitive the price is to change in the inputs. These sensitivities, commonly referred to as Greeks, play a crucial role in the financial industry by enabling analysts to measure and manage the risks to which their portfolios are exposed. Greeks also help identify and monitor pricing errors that may become increasingly significant in calibration or risk-management applications.

A classical reference point is the Black-Scholes model [2]. It gives an explicit formula for European option prices in a simplified setting where volatility is assumed to be fixed. Here, volatility can be understood as the parameter that controls the intensity of the asset price fluctuations in the model. The model is still central because it provides a common reference for comparing option prices across strikes and maturities. In later chapters, this idea will also be connected to the market convention of expressing option prices through an equivalent volatility level. The limitation of Black-Scholes is that a single fixed volatility is usually not flexible enough to represent option prices across different strikes and maturities.

This work uses the Heston model [3] because it relaxes this constant-volatility assumption while keeping the problem numerically tractable. In Heston, the variance of

the asset is itself modelled as a random process. This allows the model to represent richer price patterns than Black-Scholes, especially when prices vary across strikes and maturities. At the same time, Heston keeps enough mathematical structure to be priced numerically by the Fourier-COS method introduced in Chapter 2. This extra flexibility also makes calibration more difficult. In this context, calibration means choosing the model parameters so that the model reproduces a given set of option quotes. Under Black-Scholes, the main volatility input is much simpler to handle; under Heston, several interacting parameters must be fitted at the same time.

A standard way to compute the price of derivatives when no analytical solution or other numerical methods are available are Monte Carlo methods [4]. The idea is to simulate many possible future paths of the underlying asset and average the resulting discounted payoffs. This approach is flexible because it can be adapted to many payoff structures and model dynamics. However, the probabilistic nature of Monte Carlo methods leads to statistical errors that can only be reduced through a large number of simulations [5], making these methods computationally expensive in repeated-use settings.

Explicit pricing formulas suitable for deterministic methods such as COS are available only in restricted cases. This thesis deals with one of those favored cases: European put options, where deterministic Heston-COS reference is more appropriate. The experiments we are going to perform require repeated pricing calls, implied-volatility generation, calibration tests, and Greek benchmarks This repetition can make the numerical experiment considerably more expensive, because the reference pricer is called many times across contract grids, calibration routines, and sensitivity diagnostics. This repeated-use setting is where neural approximations become attractive.

A neural network can be trained to approximate a numerical pricing routine and then evaluated very quickly. The idea is to pay the cost of generating accurate reference data once, and then train a model that can be evaluated much faster inside calibration or sensitivity workflows. This is the motivation behind neural pricing and calibration approaches such as [6].

However, speed alone is not sufficient in a financial workflow. A learned component is useful only if its errors can be measured, if its weak regions can be detected and if its outputs remain reliable when used in calibration or sensitivity analysis. The question this thesis aims to answer is not whether neural networks can be fast, but whether they can be assessed in a way that is meaningful for a pricing workflow. The next section makes this context precise by defining the benchmark problem, the role of the Heston model, and the downstream task used to evaluate reliability

1.2. Problem Setting

A put option gives value to its holder when the underlying asset finishes below the strike price. In the European case considered here, the option is evaluated only at the maturity

date. This setting is deliberately restricted: the payoff rule is simple, the exercise date is fixed, and the numerical reference can be built under controlled assumptions. At the same time, the model is expressive enough to generate non-trivial behaviour across strikes, maturities, and volatility regimes.

The problem is therefore not the complexity of the contract itself, but the reliability of the computational workflow built around it. The project studies three connected tasks: approximating the outputs of the Heston pricing routine, using those approximations inside calibration of the model parameters, and evaluating the quality of the resulting sensitivities. Each of these tasks can fail in a different way. A neural surrogate may have a small average pricing error but still introduce structured errors across moneyness[1] or maturity. A calibration routine may fit observed quotes while recovering unstable parameter values. A differentiable pricing model may produce reasonable prices while giving inaccurate Greeks.

This is especially relevant in regions where the pricing surface, understood here as the option price as a function of moneyness and maturity, changes rapidly. The most delicate cases in this thesis are short maturities and near-the-strike contracts, where the current underlying price is close to the strike. This ATM region is especially relevant because small movements in the underlying can produce large changes in local curvature. For a European put, the payoff changes slope abruptly at the strike: below K it increases as the asset price falls, while above K it is zero. This sharp corner is often called a payoff kink. Near maturity, it makes the pricing surface harder to approximate, especially for second-order sensitivities such as Gamma.

For this reason, the thesis treats benchmarking as part of the problem itself. Neural components are not evaluated only by speed or by global error metrics, but also by their behaviour in the downstream tasks for which they are intended. The central question is whether these components can be useful inside a Heston pricing workflow without hiding errors that later affect calibration or Greek computation.

1.3. Methodological Approach and Contributions

The proposed methodology is built around a benchmarked Heston workflow that first constructs a deterministic numerical reference and then evaluates neural approximation methods against it. This numerical reference has two roles: it provides accurate option values and Greek benchmarks, and it generates the controlled data used to train and test the machine-learning components. This makes it possible to evaluate neural methods under explicit numerical conditions and to distinguish computational acceleration from genuine loss of reliability.

[1] Moneyness, denoted by m, measures the relative position of the underlying price with respect to the strike. For the European put options considered in this thesis, at-the-money (ATM) contracts have an underlying price close to the strike, in-the-money (ITM) contracts have an underlying price below the strike, and out-of-the-money (OTM) contracts have an underlying price above the strike.

Using this reference as the common benchmark, we study two neural network approaches. The first is a supervised artificial neural network (ANN) surrogate trained to approximate the outputs of the Heston pricing routine. This surrogate is then used inside a calibration routine, where the objective is to recover model parameters efficiently from option quotes. The second approach is a physics-informed neural network (PINN) used to approximate the option price surface directly from the Heston pricing equation. Since this price surface is differentiable, it can also be used to compute Greeks though automatic differentiation [7]. The complete structure of this benchmarked workflow is summarized in Figure 1.1.

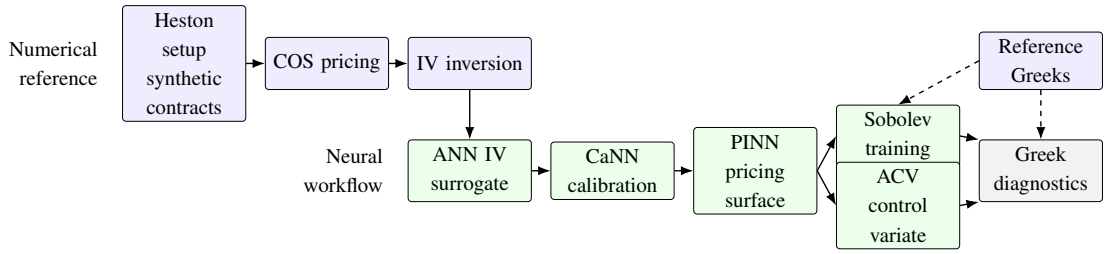


Fig. 1.1. High-level structure of the benchmarked Heston workflow. Solid arrows show the main computational flow; dashed arrows show reference Greeks used to compute diagnostic errors and to define Sobolev Greek targets.

A particular focus is placed on the quality of these Greeks. Sensitivity computation is therefore not treated as a secondary output of the pricing model, but as a separate diagnostic problem. For this reason, the final part of the workflow studies two complementary ways of improving or analysing Greek accuracy. One introduces derivative information during training, while the other uses an analytic reference component to correct or diagnose the learned sensitivities. These two ideas are later developed as the Sobolev-enhanced and analytic-control-variate experiments.

The main contribution is the construction and evaluation of this full workflow under a common benchmark. The ANN calibration block is used to study whether neural surrogates can accelerate repeated Heston evaluations and parameter recovery. The PINN block is used to study whether a neural price surface can provide reliable prices and sensitivities at the same time. The final experiments analyse the regions where Greek estimation is most difficult and assess whether the two Greek-improvement strategies reduce those errors.

This organization is important for the interpretation of the results. The ANN is judged not only as a fast function approximator, but also as the forward engine used by the calibration routine. The PINN is judged not only by price error, but also by spatial error structure and derivative quality. The Greek analysis is treated as a separate problem because automatic differentiation through a learned price surface does not by itself guarantee reliable sensitivities.

1.4. State of the Art

The literature relevant to this thesis sits at the intersection of numerical option pricing, model calibration, and neural approximation. Classical model-based pricing provides the financial foundation. The Black–Scholes model gives a closed-form benchmark under constant volatility [2], while the Heston model extends this framework by introducing stochastic variance and semi-analytical pricing formulas [3]. Simulation-based approaches, especially Monte Carlo methods, are also a central reference in derivative pricing because of their flexibility across payoffs and dynamics [4]. Their main drawback in the present setting is that repeated simulation can be expensive and introduces sampling noise, which is inconvenient when the objective is to build deterministic benchmarks for calibration and Greek diagnostics.

Fourier and spectral methods provide another important line of work. When the characteristic function of the model is available, transform methods can price options accurately without simulating paths. FFT-based pricing [8] and the Fourier-COS method [9] are standard examples of this approach. The COS method is especially useful here because it gives a deterministic and accurate reference layer for repeated pricing, implied-volatility generation, and benchmark construction. Its performance, however, still depends on numerical choices such as the truncation interval, the number of expansion terms, and the stability of the implied volatility inversion.

Neural methods enter this landscape as acceleration and approximation tools. Feed-forward ANN surrogates can learn pricing or implied-volatility maps from synthetic labels and then evaluate them much faster than the numerical solver, which makes them attractive for calibration workflows [6]. Physics-informed neural networks follow a different route: instead of learning only from labels, they use the pricing equation itself as part of the training objective. Recent work applies this idea to Heston-type option pricing and related PDE settings [10], [11]. More recent operator-learning work for stochastic-volatility models also reports a useful warning for the present thesis: highly accurate Heston prices can still lead to noisy autodiff Greeks, especially near the at-the-money region [12]. At the same time, the PINN literature reports well-known difficulties with optimization, loss balancing, gradient flow, and robustness across the training domain [13], [14], [15].

For Greek computation, automatic differentiation provides a natural way to obtain sensitivities from differentiable neural price surfaces [7], although accurate prices do not automatically imply accurate derivatives. This motivates derivative-aware training strategies such as Sobolev-type supervision [16], deep differential learning of Heston parameter sensitivities [17], and broader derivative-informed operator-learning methods for financial surfaces [18]. These ideas are complemented here by correction and diagnostic tools based on analytic control variates, which are related to the variance-reduction tradition in Monte Carlo methods [5].

Table 1.1 summarizes these methodological blocks and clarifies their role in this the-

sis. The table also motivates the structure of the proposed workflow: COS pricing is kept as the deterministic reference, ANN and CaNN components are used for fast calibration, and PINN-based methods are evaluated as differentiable pricing surfaces whose Greek quality must be diagnosed separately.

TABLE 1.1. METHODOLOGICAL BLOCKS AND ROLE IN THIS THESIS

Block	**Representative methods**	**Main strengths**	**Main limitations**
Classical model-based pricing	Black–Scholes, Heston semi-closed pricing [2], [3]	Financial interpretability; analytical and semi-analytical pricing structure	Constant-volatility benchmark is too simple; Heston calibration is more involved
Spectral pricing	FFT/Carr–Madan, COS/Fang–Oosterlee [8], [9]	High accuracy with efficient repeated pricing	Requires careful numerical setup
Data-driven surrogates	ANN pricers/calibrators [6]	Very fast inference once trained	Domain-shift and extrapolation risk
Physics-informed surrogates	PINN/gPINN and physics-informed operator variants [10], [11], [12]	Injects structural constraints into learning	Training instability, sensitive optimization, and noisy autodiff Greeks near difficult regions [12], [13], [14], [15]
Greek-improvement and correction methods	Automatic differentiation, Sobolev training, differential learning, control variates [5], [7], [16], [17], [18]	Direct sensitivity computation; targeted reduction of derivative errors	Derivative accuracy depends on surface smoothness; control variates are problem-specific

Source: Author's synthesis from the cited literature

1.5. Scope and Thesis Structure

The scope is intentionally controlled. All experiments are carried out for European vanilla options under the Heston model, using synthetic data and numerical references whose set-

tings can be inspected. This restriction is appropriate for the objective of the work: to test whether neural components can accelerate pricing, calibration, and sensitivity computation while remaining comparable to established numerical methods.

The remainder of the thesis is organized as follows. Chapter 2 introduces the mathematical and financial background needed for the rest of the work, including option pricing, the Heston model, Greeks, and the numerical methods used as referece. Chapter 3 develops the supervised neural methods for Heston pricing and calibration: the ANN surrogate pricer and the CaNN inverse workflow. Chapter 4 introduces the PINN framework for Heston pricing and sensitivity computation, including the extensions used to study Greeks accuracy. Chapter 5 reports the numerical benchmarks for pricing, calibration, PINN performance, and Greek diagnostics. Chapter 6 summarizes the main conclusions and discusses possible extensions.

2. MATHEMATICAL AND FINANCIAL BACKGROUND

2.1. Mathematical Framework For Option Pricing

This section fixes the common mathematical setup used in the rest of the thesis for European put pricing.

Following standard stochastic-process notation [19, Ch. 1], we work on a filtered probability space

$$(\Omega, \mathcal{F}, (\mathcal{F}_t)_{t\in[0,T]}, \mathbb{P}).$$

Here, Ω is the set of possible outcomes, $\mathcal{F}$ is the collection of measurable events, $\mathbb{P}$ is the original probability measure, and $\mathcal{F}_t$ represents the information available up to time t. The filtration is increasing: if $s \le t$, then $\mathcal{F}_s \subseteq \mathcal{F}_t$, so information is accumulated over time.

The maturity is finite, $T > 0$. Time is continuous, $t \in [0, T]$, and the time to maturity is $\tau = T - t \in [0, T]$. The underlying asset price $S = (S_t)_{t\in[0,T]}$ is assumed to be positive and adapted to the filtration, meaning that S_t is observable with the information available at time t. The strike $K > 0$ and the risk-free rate $r \ge 0$ are constant. When a fixed valuation date is needed, it is denoted by $t_0 \in [0, T)$ and $S_0 := S_{t_0}$. When it is useful to distinguish the two uses of moneyness, we write $m_0 := S_0/K$ for the fixed-date moneyness used in quote grids and $m := S/K$ for the generic moneyness state used in PDE formulations. Thus, m_0 denotes the coordinate observed at the valuation date, while m denotes the state variable over which the pricing equation is written.

The contract studied throughout the numerical pipeline is a European vanilla put. Its terminal payoff is

$$\Psi_{\text{put}} = (K - S_T)^+, \qquad (x)^+ = \max(x, 0). \tag{2.1}$$

The payoff changes slope at the strike, $S_T = K$. This non-differentiable point is often referred to as the payoff kink. Figure 2.1 shows the normalized put payoff Ψ_{put}/K as a function of the normalized terminal price S_T/K.

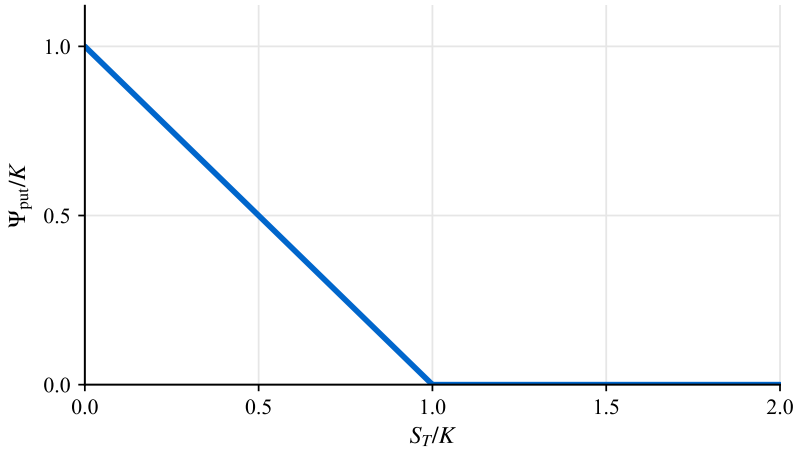


Fig. 2.1. Normalized terminal payoff of a European put. The holder has the right, but not the obligation, to exercise at maturity; therefore the payoff is positive only when $S_T/K < 1$ and is zero for $S_T/K \geq 1$.

A global notation glossary is provided in Appendix D and will be extended as new symbols are introduced.

For valuation, assume that there exists an equivalent risk-neutral measure $\mathbb{Q}$ under which discounted tradable prices are martingales. The standard risk-neutral valuation formula for the European payoffs considered in this thesis is [19, Ch. 8]

$$V_t = e^{-r(T-t)}\,\mathbb{E}^{\mathbb{Q}}[\Psi \mid \mathcal{F}_t] = e^{-r\tau}\,\mathbb{E}_t^{\mathbb{Q}}[\Psi], \qquad 0 \leq t \leq T. \tag{2.2}$$

Here, V_t is the option value at time t, $\mathbb{E}_t^{\mathbb{Q}}[\cdot]$ denotes the conditional expectation given $\mathcal{F}_t$, and in the put case $\Psi = \Psi_{\text{put}}$. The risk-neutral measure is the pricing probability measure under which discounted tradable asset prices have no expected excess return. This is why future payoffs can be evaluated by taking conditional expectations under $\mathbb{Q}$ and discounting at the risk-free rate r.

In the following sections, this framework is connected to three ingredients used throughout the thesis: the stochastic-calculus tools needed for the model equations, the Heston stochastic-volatility dynamics, and the characteristic-function/Fourier-COS pricing route.

2.2. Stochastic Calculus Background

This section introduces only the stochastic-calculus notions needed by the pricing methods used later. The presentation follows standard probability definitions and the transform-pricing notation adopted in [19, Ch. 1] and [9, Sec. 2].

The Heston dynamics introduced below are written as stochastic differential equations (SDEs), so they require a continuous-time source of randomness. Following standard SDE notation [19, Chs. 2–4], a Brownian motion, also called a Wiener process, is an adapted process W_t with $W_0 = 0$, continuous paths, independent increments, and $W_t - W_s \sim$

$\mathcal{N}(0, t-s)$ for $0 \leq s < t \leq T$. In stochastic differential equations, dW_t denotes the infinitesimal random component generated by this process. If two Brownian motions are correlated, their instantaneous quadratic covariation is written as $d\langle W^{(1)}, W^{(2)} \rangle_t = \rho\, dt$, with $\rho \in [-1, 1]$.

For a real random variable X, the cumulative distribution function is

$$F_X(x) = \mathbb{P}(X \leq x). \tag{2.3}$$

Here, x is a real evaluation point and $\mathbb{P}$ denotes probability. If F_X is differentiable, the associated probability density function is

$$f_X(x) = \frac{\partial F_X(x)}{\partial x}. \tag{2.4}$$

Following this notation, the characteristic function of X is

$$\phi_X(u) := \mathbb{E}\left[e^{iuX}\right] = \int_{-\infty}^{\infty} e^{iux} dF_X(x) = \int_{-\infty}^{\infty} e^{iux} f_X(x) dx, \tag{2.5}$$

for $u \in \mathbb{R}$, where i is the imaginary unit.

Applying the logarithm to ϕ_X, we obtain the cumulant characteristic function

$$\zeta_X(u) = \log \mathbb{E}\left[e^{iuX}\right] = \log \phi_X(u). \tag{2.6}$$

Its Maclaurin expansion is

$$\zeta_X(u) = \sum_{k=0}^{\infty} \frac{\partial^k \zeta_X(u)}{\partial u^k}\bigg|_{u=0} \frac{u^k}{k!} = \sum_{k=0}^{\infty} \zeta_k \frac{u^k}{k!}, \tag{2.7}$$

where $\zeta_k \equiv \frac{\partial^k \zeta_X(u)}{\partial u^k}\Big|_{u=0}$ is the k-th cumulant.

The moment generating function is

$$\mathcal{M}_X(u) := \phi_X(-iu) = \mathbb{E}\left[e^{uX}\right]. \tag{2.8}$$

Using $\mathcal{M}_X(u) = \phi_X(-iu)$ and $\zeta_X(u) = \log \mathcal{M}_X(u)$, cumulants can be computed from the characteristic function as

$$\zeta_k = \frac{\partial^k \zeta_X(u)}{\partial u^k}\bigg|_{u=0} = \frac{\partial^k}{\partial u^k} \log \phi_X(-iu)\bigg|_{u=0}. \tag{2.9}$$

These objects are central in the rest of this chapter: the characteristic function provides direct access to Fourier-domain pricing formulas, and low-order cumulants are later used to construct truncation intervals for the COS method.

2.3. The Heston Stochastic Volatility Model

Before introducing stochastic volatility, it is useful to fix the implied-volatility convention used throughout the thesis. Black–Scholes is the constant-volatility reference model used for this convention [1], [2]. In the no-dividend setting considered here, the Black–Scholes price of a European put is

$$V_{BS}^{\text{put}}(S_0, K, r, \tau, \sigma) = Ke^{-r\tau}\Phi(-d_2) - S_0\Phi(-d_1), \tag{2.10}$$

where Φ is the cumulative distribution function of a standard normal random variable and

$$d_1 = \frac{\log(S_0/K) + (r + \frac{1}{2}\sigma^2)\tau}{\sigma\sqrt{\tau}}, \qquad d_2 = d_1 - \sigma\sqrt{\tau}. \tag{2.11}$$

Given a target European put value V_{target}, its implied volatility is the value σ_{imp} that reproduces this price when inserted into the same Black–Scholes formula:

$$V_{BS}^{\text{put}}(S_0, K, r, \tau, \sigma_{\text{imp}}) = V_{\text{target}}. \tag{2.12}$$

Thus, implied volatility is a transformation of prices into volatility units; it is not a structural parameter of the Heston model. It is nevertheless the main comparison space in this thesis, because option quotes and calibration errors are usually interpreted across moneyness and maturity in implied-volatility units. The numerical inversion used to compute σ_{imp} is detailed in Section 2.6.

The Heston model extends this constant-volatility benchmark by making variance stochastic. In the risk-neutral pricing form used in this thesis, the dynamics are given by the following system of coupled stochastic differential equations:

$$\begin{cases} dS_t = rS_t\,dt + \sqrt{v_t}\,S_t\,dW_t^{(S,\mathbb{Q})}, \\ dv_t = \kappa(\bar{v} - v_t)\,dt + \gamma\sqrt{v_t}\,dW_t^{(v,\mathbb{Q})}, \\ d\langle W^{(S,\mathbb{Q})}, W^{(v,\mathbb{Q})}\rangle_t = \rho\,dt. \end{cases} \tag{2.13}$$

Here, S_t is the underlying price, v_t is the instantaneous variance, and $W_t^{(S,\mathbb{Q})}$ and $W_t^{(v,\mathbb{Q})}$ are Brownian motions under the risk-neutral measure $\mathbb{Q}$. In this background chapter we write the variance process as v_t. Later, in the implementation-oriented chapters, the same variance state is denoted by v_t; in particular, the initial variance at the valuation date is $v_0 = v_{t_0}$ here and v_0 in the dataset notation. The variance equation is a square-root, or CIR-type, diffusion: it is mean-reverting toward the long-run level $\bar{v}$ at speed κ, and its random variation has instantaneous scale $\gamma\sqrt{v_t}$. Thus, γ is the volatility of variance, often called vol-of-vol.

The model parameters are summarized in Table 2.1. The table keeps the interpretation operational: each row states how the parameter enters the dynamics and which part of the generated price or IV grid it most directly affects.

TABLE 2.1. INTERPRETATION OF HESTON PARAMETERS

Parameter	Role in the dynamics	Main practical effect
ρ	Instantaneous correlation between the Brownian drivers of price and variance.	Changes the asymmetry of prices and IV values across moneyness.
γ	Volatility of the variance process, or vol-of-vol.	Controls the amplitude of random variation in the variance path.
$\bar{\nu}$	Long-run mean level of the variance process.	Sets the level toward which variance reverts.
ν_0	Initial variance level at valuation time (ν_{t_0}).	Sets the starting variance state, which is especially relevant for short maturities.
κ	Speed of mean reversion of ν_t toward $\bar{\nu}$.	Controls how quickly variance moves back toward its long-run level.

Source: Based on [3] and [20, Chs. 2–3]

A common positivity condition for the CIR variance process is the Feller condition

$$2\kappa\bar{\nu} \geq \gamma^2. \tag{2.14}$$

In practice, calibrated parameter sets may violate this inequality while still producing usable market fits; therefore, it is treated as a diagnostic and stability indicator rather than as a hard calibration constraint in many numerical workflows [20, Chs. 2–3].

The other output quantities used later are Greeks. To keep them distinct from Heston parameters such as γ and ρ, Greek names are written in roman text. For a value function $V(S, \nu, t; r)$, we use

$$\begin{aligned} \text{Delta} &= \frac{\partial V}{\partial S}, & \text{Gamma} &= \frac{\partial^2 V}{\partial S^2}, \\ \text{Vega} &= \frac{\partial V}{\partial \nu}, & \text{Rho} &= \frac{\partial V}{\partial r}, \\ \text{Theta} &= \frac{\partial V}{\partial t} = -\frac{\partial V}{\partial \tau}. \end{aligned} \tag{2.15}$$

Delta is the first-order sensitivity to the underlying price. Gamma is the second-order sensitivity to the underlying price. In this thesis, Vega denotes sensitivity with respect to the Heston variance state; in constant-volatility models Vega is often defined with respect to volatility instead. Rho is the sensitivity to the risk-free rate, and Theta is the sensitivity to calendar time, equivalently the negative derivative with respect to time to maturity.

2.4. Pricing Via Characteristic Functions

We now turn to the Fourier representation used for pricing. The relevant object is the characteristic function of a log-price state. For example, if $X_T = \log S_T$, then

$$\phi_X(u; t, S_t, \nu_t) = \mathbb{E}_t^{\mathbb{Q}}\Big[e^{iuX_T} \mid S_t, \nu_t\Big]. \tag{2.16}$$

When pricing by log-moneyness, the same definition is applied to $Y_T = \log(S_T/K)$. Under Heston dynamics, this characteristic function has a semi-closed representation [3], which is why the model is well suited to the Fourier-COS pricing route developed in the next sections.

This transform representation is useful because the density of the terminal log-state is usually not handled directly in pricing. Fourier pricing instead works with the characteristic function and recovers prices through transform integrals.

Using the density and characteristic-function notation introduced above, let $x = X(t_0)$ be the current state and $y = X(T)$ the terminal state. The discounted valuation formula can be written as

$$V(t_0, x) = e^{-r\tau} \int_{\mathbb{R}} V(T, y)\, f_X(T, y; t_0, x)\, dy. \tag{2.17}$$

Here, $V(T, y)$ is the terminal payoff as a function of the terminal state, and $f_X(T, y; t_0, x)$ is the density of $X(T) = y$ conditioned on the current state $X(t_0) = x$. The corresponding characteristic function is

$$\phi_X(u; t_0, x) := \mathbb{E}_{t_0}^{\mathbb{Q}}\Big[e^{iuX(T)} \mid X(t_0) = x\Big] = \int_{\mathbb{R}} e^{iuy} f_X(T, y; t_0, x)\, dy. \tag{2.18}$$

If $f_X(T, \cdot; t_0, x)$ exists and $\int_{\mathbb{R}} |\phi_X(u; t_0, x)|\, du < \infty$, Fourier inversion gives:

$$f_X(T, y; t_0, x) = \frac{1}{2\pi} \int_{\mathbb{R}} e^{-iuy} \phi_X(u; t_0, x)\, du. \tag{2.19}$$

Let $\hat{V}_T(u) := \int_{\mathbb{R}} e^{-iuy} V(T, y)\, dy$ denote the Fourier transform of the payoff at maturity. Substituting the inverse representation of f_X into the pricing integral and exchanging the order of integration gives

$$V(t_0, x) = e^{-r\tau} \frac{1}{2\pi} \int_{\mathbb{R}} \phi_X(u; t_0, x)\, \hat{V}_T(u)\, du. \tag{2.20}$$

Compared with PDE solvers, this approach avoids full space-time gridding. Compared with Monte Carlo, it is deterministic and does not introduce sampling noise. In repeated-pricing tasks (surface generation, calibration loops, and large synthetic datasets), this is a major practical advantage.

The Fourier-COS method used in the next section is a specific spectral discretization of this transform-based pricing framework: it replaces the continuous integral by a finite cosine expansion over a truncated interval, preserving high accuracy with efficient vectorized evaluation.

2.5. The Fourier-COS method

The previous section showed that option values can be written as transform integrals once the characteristic function of the terminal state is known. The Fourier-COS method makes this representation numerically usable by replacing the terminal-state density on a finite interval with a cosine expansion [9, Secs. 3–4]. The key point is that the cosine coefficients of this density can be computed directly from the characteristic function, so pricing reduces to evaluating the characteristic function on a fixed frequency grid and summing finitely many terms.

Its practical appeal in this thesis is that it combines spectral convergence with efficient vectorized evaluation once the model characteristic function is available.

Let $x = X(t_0)$, $y = X(T)$, and let $[a, b] \subset \mathbb{R}$, with $a < b$, be a truncation interval chosen to concentrate most of the probability mass of $X(T) \mid X(t_0) = x$. On this interval, the density of the terminal state is approximated by

$$\hat{f}_X(y) = \sum_{k=0}^{N-1}{}' \bar{F}_k \cos\left(\pi k \frac{y-a}{b-a}\right), \tag{2.21}$$

where $\hat{f}_X$ is the truncated cosine approximation of the density, $N \in \mathbb{N}$ is the number of cosine terms, $k \in \{0, \ldots, N-1\}$ indexes the expansion terms, $\sum'$ indicates that the $k = 0$ term is weighted by $1/2$, and the cosine coefficients are obtained from the characteristic function:

$$\bar{F}_k := \frac{2}{b-a} \Re\left[\phi_X\left(\frac{\pi k}{b-a}\right) \exp\left(-\frac{i\pi a k}{b-a}\right)\right]. \tag{2.22}$$

The operator $\Re[\cdot]$ denotes the real part; for example, $\Re[\alpha + i\beta] = \alpha$ for $\alpha, \beta \in \mathbb{R}$.

Starting from

$$V(t_0, x) = e^{-r\tau} \int_{\mathbb{R}} V(T, y) f_X(T, y; t_0, x)\, dy, \tag{2.23}$$

the COS discretization yields

$$V(t_0, x) \approx e^{-r\tau} \sum_{k=0}^{N-1}{}' \Re\left[\phi_X\left(\frac{\pi k}{b-a}\right) \exp\left(-\frac{i\pi a k}{b-a}\right)\right] H_k, \tag{2.24}$$

where the payoff coefficients H_k, for $k = 0, \ldots, N-1$, are

$$H_k := \frac{2}{b-a} \int_a^b V(T, y) \cos\left(\frac{y-a}{b-a}\pi k\right) dy. \tag{2.25}$$

For plain-vanilla options, H_k has closed-form expressions, so the numerical effort is concentrated in evaluating the characteristic function on a fixed frequency grid [9, Sec. 3].

The choice of $[a, b]$ controls truncation error. Following the cumulant-based rule in [9, Sec. 5], we use

$$[a, b] = \left[c_1 - L\sqrt{c_2 + \sqrt{c_4}},\ c_1 + L\sqrt{c_2 + \sqrt{c_4}}\right], \tag{2.26}$$

with c_1, c_2, c_4 the first, second, and fourth cumulants of $X(T)$ and $L > 0$ a scaling parameter. This connects directly with the cumulant machinery introduced in Section 2.2. In the numerical experiments of this thesis, the integration interval is selected using the refined cumulant-based strategy detailed in Appendix A.

The cumulant interval in Equation (2.26) addresses the state-space truncation part of the approximation, but it does not by itself control the error introduced by using only N cosine terms. It is therefore useful to separate two error sources:

$$\left|V - V_N^{\mathrm{COS}}\right| \leq \underbrace{\varepsilon_{\mathrm{space}}([a,b])}_{\text{state-space truncation}} + \underbrace{\varepsilon_{\mathrm{series}}(N;[a,b])}_{\text{finite COS series}}. \tag{2.27}$$

Here, V_N^{COS} is the COS approximation using N terms, $\varepsilon_{\mathrm{space}}$ is the error from replacing the full state space $\mathbb{R}$ by the interval $[a,b]$, and $\varepsilon_{\mathrm{series}}$ is the error from keeping only finitely many cosine terms. Increasing L reduces $\varepsilon_{\mathrm{space}}$ but may require larger N to keep $\varepsilon_{\mathrm{series}}$ small; reducing L has the opposite effect. In practice, we choose (L, N) jointly and verify stability, especially for extreme strikes and short maturities, where numerical artifacts appear first.

Implementation is fully vectorized: frequencies $u_k = \pi k/(b-a)$, exponential phase factors, and payoff coefficients are precomputed once per maturity, then reused across strike grids. This is one of the reasons COS is well suited for repeated-pricing workflows such as calibration loops and synthetic dataset generation.

2.5.1. The COS Method Applied To The Heston Model

Following the notation of Sections 2.4–2.5 and [9, Sec. 3], define the COS frequencies and log-moneyness variables as

$$u_k := \frac{k\pi}{b-a}, \quad k = 0, \ldots, N-1, \qquad x := \log\!\left(\frac{S_0}{K}\right), \qquad y := \log\!\left(\frac{S_T}{K}\right).$$

Here, x and y are log-moneyness variables: they measure the underlying price relative to the strike before and at maturity. Positive log-moneyness means the underlying is above the strike, while negative log-moneyness means it is below the strike. Under Heston dynamics, the characteristic function of y can be written in semi-closed form [3], [20]

$$\varphi_H(u;\tau,x,v_0) = \exp(C(u,\tau) + D(u,\tau)v_0 + iux)\,, \tag{2.28}$$

with

$$\begin{aligned}
d(u) &= \sqrt{(\kappa - i\rho\gamma u)^2 + \gamma^2\,(u^2 + iu)},\\
g(u) &= \frac{\kappa - i\rho\gamma u - d(u)}{\kappa - i\rho\gamma u + d(u)},\\
C(u,\tau) &= iur\tau + \frac{\kappa\bar{\nu}}{\gamma^2}\left[(\kappa - i\rho\gamma u - d(u))\,\tau - 2\log\!\left(\frac{1 - g(u)e^{-d(u)\tau}}{1 - g(u)}\right)\right],\\
D(u,\tau) &= \frac{\kappa - i\rho\gamma u - d(u)}{\gamma^2}\left(\frac{1 - e^{-d(u)\tau}}{1 - g(u)e^{-d(u)\tau}}\right).
\end{aligned} \tag{2.29}$$

Here, φ_H denotes the Heston characteristic function of the log-moneyness state, while C, D, d, and g are complex-valued auxiliary functions of the Fourier variable u and maturity τ. This is the standard semi-closed representation used in Fourier pricing; numerically, consistency in branch selection for $d(u)$ and the complex logarithm is required for stability [20, Chs. 2–3].

The COS price for one strike can then be written as

$$V(t_0, x) \approx Ke^{-r\tau} \sum_{k=0}^{N-1}{}' \, \Re\Big[\varphi_H(u_k; \tau, x, v_0)\, e^{-iu_k a}\Big]\, U_k, \tag{2.30}$$

where U_k are payoff-dependent COS coefficients for $k = 0, \ldots, N-1$. For plain-vanilla options:

$$U_k^{\text{call}} = \frac{2}{b-a}\left(\chi_k(0,b) - \psi_k(0,b)\right), \qquad U_k^{\text{put}} = \frac{2}{b-a}\left(-\chi_k(a,0) + \psi_k(a,0)\right), \tag{2.31}$$

with

$$\begin{aligned} \chi_k(\ell, q) &= \frac{1}{1+u_k^2}\Big[e^{q}(\cos(u_k(q-a)) + u_k \sin(u_k(q-a))) \\ &\qquad\qquad - e^{\ell}(\cos(u_k(\ell-a)) + u_k \sin(u_k(\ell-a)))\Big], \\ \psi_k(\ell, q) &= \begin{cases} q - \ell, & k = 0, \\ \dfrac{b-a}{k\pi}\left[\sin(u_k(q-a)) - \sin(u_k(\ell-a))\right], & k \geq 1. \end{cases} \end{aligned} \tag{2.32}$$

Here, χ_k and ψ_k are analytical payoff integrals over generic bounds ℓ and q satisfying $a \leq \ell < q \leq b$. In the implementation of this thesis, data generation is run on European puts as a numerical convention, mainly associated with the COS-pricing setup used in the project configuration.

This subsection establishes the analytical Heston-COS pricing block used as the reference framework in the following chapters.

2.5.2. Semi-Analytical Greeks from the Heston Characteristic Function

For validation purposes, the option Greeks defined in Section 2.3 can also be computed directly from the Heston characteristic-function pricing representation. A standard presentation is often written for a European call [3] and is developed in [20, Chapter 1]

$$V^{\text{call}}(S_0, K, \tau) = S_0\, \Pi_1 - Ke^{-r\tau}\, \Pi_2, \tag{2.33}$$

where $V^{\text{call}}(S_0, K, \tau)$ is the European call price and Π_1, Π_2 are risk-neutral probability-type terms in the Heston pricing formula:

$$\Pi_j = \frac{1}{2} + \frac{1}{\pi}\int_0^\infty \Re\left(\frac{e^{-iu\log K}\, \Psi_j(u)}{iu}\right) du, \qquad j \in \{1, 2\}, \tag{2.34}$$

with

$$\Psi_2(u) = \phi_X(u), \qquad \Psi_1(u) = \frac{\phi_X(u-i)}{\phi_X(-i)}, \tag{2.35}$$

where j selects the two probability terms, Ψ_j are the corresponding Fourier integrands, and ϕ_X is the characteristic function of $X_T = \log S_T$ under Heston [3]; see also [20, Chapter 1]. For puts, the same characteristic-function machinery is used through the put payoff coefficients or, equivalently, through put-call parity.

Let p denote a generic model/input variable ($r, \tau, v_0, \rho, \kappa, \gamma, \bar{v}$, etc.). If $\partial_p \Psi_j(u)$ exists and the differentiated integrand is integrable on $(0, \infty)$, differentiation under the integral sign yields [20, Chapter 11]

$$\partial_p \Pi_j = \frac{1}{\pi} \int_0^\infty \Re\left(\frac{e^{-iu \log K}}{iu} \partial_p \Psi_j(u) \right) du. \tag{2.36}$$

Therefore, Greeks are obtained by combining the derivatives of Π_1, Π_2 through the pricing identity [20, Chapter 11]. For example:

$$\text{Delta}_{\text{call}} = \partial_{S_0} V^{\text{call}} = \Pi_1 + S_0 \, \partial_{S_0} \Pi_1 - K e^{-r\tau} \partial_{S_0} \Pi_2, \tag{2.37}$$

$$\text{Gamma}_{\text{call}} = \partial^2_{S_0} V^{\text{call}} = 2\partial_{S_0} \Pi_1 + S_0 \partial^2_{S_0} \Pi_1 - K e^{-r\tau} \partial^2_{S_0} \Pi_2, \tag{2.38}$$

$$\text{Vega}_{\text{call}} = \partial_{v_0} V^{\text{call}} = S_0 \partial_{v_0} \Pi_1 - K e^{-r\tau} \partial_{v_0} \Pi_2, \tag{2.39}$$

$$\text{Rho}_{\text{call}} = \partial_r V^{\text{call}} = S_0 \partial_r \Pi_1 + K\tau e^{-r\tau} \Pi_2 - K e^{-r\tau} \partial_r \Pi_2, \tag{2.40}$$

$$\text{Theta}_{\text{call}} = \partial_t V^{\text{call}} = -\partial_\tau V^{\text{call}}. \tag{2.41}$$

The thesis focuses on European puts, so the corresponding quantities are obtained through put-call parity,

$$V^{\text{put}} = V^{\text{call}} - S_0 + K e^{-r\tau}. \tag{2.42}$$

This gives

$$\begin{aligned} &\text{Delta}_{\text{put}} = \text{Delta}_{\text{call}} - 1, \qquad \text{Gamma}_{\text{put}} = \text{Gamma}_{\text{call}}, \qquad \text{Vega}_{\text{put}} = \text{Vega}_{\text{call}}, \\ &\text{Rho}_{\text{put}} = \text{Rho}_{\text{call}} - \tau K e^{-r\tau}, \qquad \text{Theta}_{\text{put}} = \text{Theta}_{\text{call}} + rKe^{-r\tau}. \end{aligned} \tag{2.43}$$

The practical advantage is that all sensitivities are expressed in terms of derivatives of the same characteristic-function integrands. Equation (2.28) shows that the Heston characteristic function is built from $C(u, \tau)$, $D(u, \tau)v_0$, and the log-state term. Therefore, parameter derivatives can be obtained by differentiating these components and inserting the resulting integrands into the formulas above.

2.6. Implied Volatility And Numerical Inversion

Implied volatility was defined in Section 2.3 as the Black–Scholes volatility that reproduces a given target option value. This section focuses on the numerical inversion used to compute it for market quotes or model-generated prices.

Numerically, we solve the scalar nonlinear equation

$$F(\sigma) := V_{BS}^{\text{put}}(S_0, K, r, \tau, \sigma) - V_{\text{target}} = 0, \qquad \sigma > 0. \tag{2.44}$$

For European vanilla options, V_{BS} is strictly increasing in σ when $\tau > 0$, so the root is unique whenever V_{target} satisfies static no-arbitrage bounds.

The inversion routine used in this thesis is the bracketed Brent method [21, Ch. 9]. Brent's method keeps a root bracket and combines bisection with secant and inverse-quadratic interpolation steps, falling back to bisection when the interpolated step is not reliable. This choice is robust for the one-dimensional IV root and avoids derivative-sensitivity issues in low-vega regions. As an alternative, the least-squares method of Levenberg and Marquardt [22], [23] was also tested in preliminary experiments, but it showed lower numerical stability in this workflow; therefore, it is not used in the final pipeline.

Stopping is based on standard price and parameter tolerances:

$$|F(\sigma_n)| \le \varepsilon_{\text{price}} \quad \text{or} \quad |\sigma_{n+1} - \sigma_n| \le \varepsilon_\sigma, \qquad n \le n_{\max}. \tag{2.45}$$

Here, σ_n is the volatility iterate at step n, $\varepsilon_{\text{price}}$ and ε_σ are stopping tolerances, and $n_{\max}$ is the maximum number of iterations. If convergence is not reached, or if the target price violates arbitrage bounds, the point is flagged as invalid for IV reporting. In edge cases near the IV floor, the pricing stage may be recomputed with a wider COS truncation interval before retrying inversion.

The benchmark metrics used to evaluate the resulting IV, price, and Greek quantities are defined in Section 5.1.1.

3. SUPERVISED NEURAL METHODS FOR HESTON PRICING AND CALIBRATION

The main objective of this chapter is to construct the supervised neural block used in the thesis for Heston pricing and calibration. The first component is an ANN-based surrogate pricer for European vanilla options under the Heston stochastic-volatility model. Its target quantity is implied volatility, and the surrogate is designed to approximate the forward map

$$(\Theta, m, \tau, r) \mapsto \sigma_{imp}.$$

The second component is the neural calibration workflow: the trained pricer is frozen and used as a fast evaluator inside an inverse problem that recovers Heston parameters from an implied-volatility quote surface.

Reference pricing based on the numerical methods introduced in Chapter 2 is highly accurate but computationally expensive when repeated many times, as required in calibration and sensitivity workflows. In this project, the reference layer combines Heston-COS valuation (Section 2.5) with implied-volatility inversion (Section 2.6). The ANN aims to preserve pricing accuracy while reducing inference time by several orders of magnitude.

Methodologically, the chapter follows the ANN-pricing and neural-calibration framework of Liu et al. [6] as the main reference (architecture family, data ranges, and training philosophy), while explicitly documenting the implementation choices adopted in this project. The benchmark results are reported later in Chapter 5; here the focus is the construction of the supervised pricing dataset, the pricer architecture, the training protocol, and the calibration interface.

3.1. Problem Formulation

After the Heston dynamics and their COS pricing representation have been fixed in Sections 2.3 and 2.5, we now pose vanilla option pricing under Heston as a supervised regression problem. Given a dataset

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}, \qquad y_i = \sigma_{imp,i},$$

the neural network approximates the map

$$f_{NN} : \mathbb{R}^8 \to \mathbb{R}, \qquad \mathbf{x} = (\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r) \mapsto \sigma_{imp},$$

where m and τ follow the moneyness and time-to-maturity notation fixed in Chapter 2. Each label σ_{imp} is obtained from Heston-consistent synthetic prices through numerical inversion.

The training objective is to estimate parameters $\mathbf{w}$ such that the empirical prediction error over $\mathcal{D}$ is minimized. In the pricing part of this chapter, the ANN is used as a surrogate, i.e., as a fast approximation of the reference numerical pipeline in the forward map $(\Theta, m, \tau, r) \mapsto \sigma_{imp}$. In the calibration part, the same trained map is reused with the quote descriptors fixed and the Heston parameter vector varied.

We learn implied volatility instead of option price because the IV representation is directly comparable across contracts with different maturities and moneyness levels, and it is the market-standard quotation format for calibration and diagnostics. In contrast, raw prices mix scale effects (spot/strike/time) that make learning less homogeneous over the domain.

The resulting surrogate is primarily an in-domain approximator. Preliminary out-of-domain checks indicate that the network remains stable for moderate departures from the training bounds, while errors increase in extreme regimes. For this reason, the methodological core of this chapter is developed on the predefined training domain, and the extrapolation behaviour is discussed separately in the generalization and robustness analysis.

3.2. Synthetic Dataset And Supervised Learning Vocabulary

Training this surrogate requires a large set of labeled input-output pairs. To obtain a controlled and reproducible source, we generate synthetic data under the Heston model introduced in Section 2.3. Each sampled contract/model configuration is priced with the Heston-COS machinery of Section 2.5, and the resulting price is converted into the implied-volatility convention fixed in Equation (2.12) through the numerical inversion routine described in Section 2.6. Thus, the financial notation comes from the background chapter; this section only fixes the dataset vocabulary used by the implementation.

In the current pipeline, generation is organized as a *master dataset*: a table whose rows are available to several downstream workflows. Each row stores the variables needed for supervised learning and numerical diagnostics:

- The *features* are the neural-network inputs $(\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r)$.
- The *training label* is the implied volatility σ_{imp}, stored as `sigma_imp_brent`, obtained from the reference numerical pipeline.
- The *auxiliary target* is the COS price `price_cos`, retained as a reference quantity for diagnostics.
- The *quality flags* are metadata used to assess numerical reliability. They record the Feller diagnostic and the Brent-inversion status and residual used for filtering and robustness checks.

The terms *split* and *preprocessing* refer, respectively, to the train/validation/test partition and to the transformations applied before training, such as row filtering, precision changes, or feature/target normalization. This design allows ANN and PINN workflows to reuse the same synthetic source while applying experiment-specific filters only at training or evaluation time.

The raw variable set contains the Heston parameters from Equation (2.13) and the contract/market variables (K, S_0, τ, r). The background chapter uses $\bar{v}$ and v_0 for variance quantities; the dataset columns $\bar{v}$ and v_0 denote the same long-run and initial variance levels in implementation notation. We fix $K = 1$ and keep the moneyness coordinate m as the relative underlying level. This is consistent with the log-moneyness state used in the Heston-COS formula in Equation (2.28), since $x = \log(S_0/K) = \log m$. The resulting ANN input vector has eight features:

$$\mathbf{x} = (\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r) \in \mathbb{R}^8.$$

A full Cartesian grid is computationally prohibitive in this dimension, since using N points per variable scales as N^8 (already intractable for moderate N). Therefore, the synthetic inputs are sampled with Latin Hypercube Sampling (LHS), which stratifies each coordinate and provides broad marginal coverage for a prescribed number of samples [24]. In our implementation, LHS is applied directly to the full input vector with a fixed seed and a target size of 10^7 samples in the master dataset. The specific ranges, constraints, and splitting/preprocessing rules are detailed below.

3.2.1. Parameter Ranges And Constraints

The sampling domain combines Heston parameters, contract variables, and market rate. Following the ranges used in our configuration (largely aligned with [6]), the training domain is summarized in Table 3.1.

TABLE 3.1. INPUT RANGES USED FOR SYNTHETIC DATA GENERATION

Parameter	Range
ρ	$[-0.9, 0.0]$
κ	$[0.0, 3.0]$
γ	$[0.01, 0.8]$
$\bar{v}$	$[0.01, 0.5]$
v_0	$[0.05, 0.5]$
m	$[0.6, 1.4]$
τ	$[0.05, 3.0]$
r	$[0.0, 0.05]$

Source: Based on [6] and project configuration choices

The maturity range enforces $\tau > 0$, which is required by the pricing and implied-volatility conventions introduced in Chapter 2. We also record the Feller diagnostic from Equation (2.14). In the dataset, `feller_slack` stores the signed margin of that condition and `feller_ok` stores the corresponding Boolean pass/fail flag. Feller-violating points are *not* removed during generation; they are kept in the master dataset for controlled ablation studies and can be filtered later if a particular experiment requires it.

During synthesis we fix the strike to $K = 1$ and price European put options. With this convention, the quote grid is expressed through m, which reduces the dimensionality from nine variables $(\Theta, S_0, K, \tau, r)$ to eight inputs (Θ, m, τ, r) without loss of information for this setup.

3.2.2. Dataset Splitting And Preprocessing

After generation, we keep all selected rows in the master dataset and split into train/-validation/test subsets with proportions $0.8/0.1/0.1$. In the reported configuration, splits are random (without mandatory stratification), and filtering policies are deferred to each training experiment.

In the baseline configuration, no explicit feature or target normalization is applied (Liu-style setup). Therefore, the model is trained directly on the physical scale of the variables $(\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r)$ and predicts IV in absolute units.

Data quality control is integrated in the synthesis pipeline through explicit flags, not hard row deletion by default. In particular, we track Brent inversion status and the absolute price reconstruction residual associated with the IV inversion of Section 2.6. Keeping these quantities in the table makes the filtering rule auditable: a later experiment can state exactly whether it used all rows, only successful IV inversions, only Feller-admissible rows, or a stricter residual threshold. For storage efficiency, output tables are persisted in `float32`, whereas COS pricing and Brent inversion are computed internally in `float64`.

3.2.3. Numerical Label Generation (COS + Brent)

Each label is produced in two deterministic numerical steps already defined in the background chapter. First, the target option value is computed with the Heston-COS solver for the sampled feature vector. Second, that value is converted to implied volatility using the Brent inversion routine from Section 2.6. Thus, the ANN target is a Heston-consistent implied volatility, not an independently assumed Black–Scholes volatility.

In our implementation, Brent is configured with strict numerical controls (absolute tolerance 10^{-8}, bounded search interval, and iteration cap). Additionally, when the inferred volatility hits the lower IV floor, an adaptive COS fallback can be triggered by widening the truncation interval before retrying inversion. This is the same numerical-error concern discussed in Appendix A: COS labels must be stable because they are later

treated as supervised targets.

Rather than dropping those edge cases at generation time, the pipeline records quality metadata (`brent_success`, `brent_abs_residual`, and retry diagnostics) so that robustness filters can be applied transparently at training time. This preserves full traceability of numerical reliability across the domain.

3.3. Neural Network Architecture

The pricing surrogate is implemented as a fully connected neural network that approximates a smooth nonlinear map from model/contract inputs to implied volatility. This choice is consistent with the structure of the problem: inputs are low-dimensional continuous features without spatial topology or sequential dependence, and the target is a scalar regression value.

From a function-approximation perspective, a feed-forward multilayer perceptron (MLP) provides enough expressive capacity to capture the interactions between Heston parameters, moneyness, maturity, and rates on the selected domain. Compared with architectures designed for images or sequences (e.g., CNNs or transformers), an MLP is more parsimonious and easier to train for this tabular setting.

This choice is also consistent with the classical universal-approximation results: under standard assumptions, feed-forward networks with non-polynomial activations can approximate continuous functions on compact sets to arbitrary accuracy [25], [26]. In this thesis, this theorem is used as a representation-capacity motivation, while practical accuracy is established empirically through out-of-sample benchmarks.

3.3.1. Model Class

Following [6], we use a fully connected feed-forward network (FCNN). Let $\mathbf{x} \in \mathbb{R}^8$ be

$$\mathbf{x} = (\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r),$$

and let $y = \sigma_{imp} \in \mathbb{R}$ denote the target implied volatility. The model defines a parametric function

$$\hat{\sigma}_{imp} = f_{NN}(\mathbf{x}; \mathbf{w}),$$

with trainable parameters $\mathbf{w}$.

Once $\mathbf{w}$ is fixed after training, inference is deterministic: for the same input $\mathbf{x}$, the network always returns the same prediction $\hat{\sigma}_{imp}$. This property is important for downstream calibration and sensitivity analysis, where numerical consistency across repeated calls is required.

The FCNN acts as a surrogate only at inference time. Target labels are still generated with the numerical pipeline (Heston COS pricing followed by implied-volatility inversion), so the network learns to emulate that reference solver on the sampled domain.

3.3.2. Architecture Details

The selected architecture used in the final pipeline is

$$8 \to 200 \to 200 \to 200 \to 200 \to 1,$$

with `tanh` activation in hidden layers and linear activation in the output layer. In compact form,

$$h^{(1)} = \phi(W^{(1)}x + b^{(1)}), \quad \ldots, \quad h^{(4)} = \phi(W^{(4)}h^{(3)} + b^{(4)}), \quad \hat{y} = W^{(5)}h^{(4)} + b^{(5)},$$

where $\phi(\cdot) = \tanh(\cdot)$.

Weights are initialized with Xavier uniform initialization and biases are initialized to zero. In the baseline setup, dropout is disabled ($p = 0$) and no batch normalization layers are used. This configuration follows a controlled, reproducible design intended to isolate the impact of optimization and data quality choices.

TABLE 3.2. NEURAL NETWORK ARCHITECTURE USED FOR THE ANN PRICER

Hyperparameter	**Value**
Input dimension	8
Hidden layers	4
Hidden width	200 neurons per layer
Hidden activation	`tanh`
Output dimension	1
Output activation	Linear
Weight initialization	Xavier uniform
Bias initialization	Zeros
Dropout	$p = 0.0$
Batch normalization	Disabled

Source: Based on [6], with selected `tanh` activation

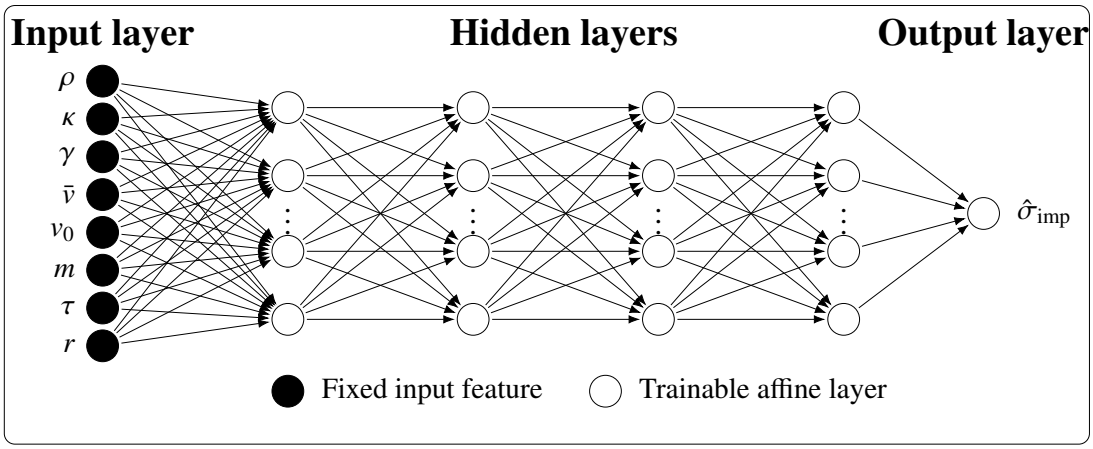


Fig. 3.1. Fully connected ANN pricer used in the forward Heston implied-volatility surrogate. The eight fixed inputs are mapped through four `tanh` hidden layers of width 200, and the final linear layer returns the predicted implied volatility.

arreglar fig. 4.1 -> hay una caja atravesada por la línea

3.3.3. Output Interpretation

The network output is a scalar implied volatility in annualized units (decimal form), not an option price. Therefore, prediction errors are interpreted directly in IV space, which is the representation used in the calibration workflow and in most volatility-surface diagnostics.

Because the output layer is linear, the model is not hard-constrained to produce strictly positive values for every possible input. In practice, training on a physically admissible domain with positive IV labels strongly reduces this risk inside the sampled region. For downstream use, any out-of-domain prediction can be explicitly checked and, if required, filtered before subsequent numerical steps.

This output definition also preserves differentiability of the surrogate with respect to inputs, which is useful in later chapters when the ANN pricer is embedded in calibration and sensitivity pipelines.

3.4. Training Procedure

Training the network means adjusting the weights $\mathbf{w}$ so that the empirical loss on the supervised dataset decreases. At this stage we describe the common training loop; the numerical rule used to update the weights, and the meaning of optimizer in this setting, are introduced in Section 3.4.2.

The training loop is fully configuration-driven and follows an epoch-based supervised learning scheme. At each epoch, the configured update rule modifies the parameters using the training split, then the model is evaluated on the validation split without gradient com-

putation. This separates the learning step from the monitoring step: the former changes $\mathbf{w}$, while the latter only measures out-of-sample behaviour under the current weights.

Each epoch stores train/validation metrics, current update-rule identity, and learning rate. When enabled, the validation monitor can be either the raw validation loss or a trimmed variant that discards the top-tail hardest samples before aggregation. This provides a robust alternative when a small fraction of outlier points may dominate the monitor value.

Execution device is selected automatically (`mps` when available, otherwise `cpu`), and data-shuffling reproducibility is controlled through a fixed generator seed from configuration. During training, two checkpoints are maintained: the *last* checkpoint (always overwritten each epoch) and the *best* checkpoint (updated only when the monitored validation metric improves). The best checkpoint is used as the reference model for post-training evaluation and downstream calibration.

3.4.1. Loss Function

Let $\{(\mathbf{x}_i, \sigma_i)\}_{i=1}^{B}$ denote a mini-batch, where σ_i is the target implied volatility and $\hat{\sigma}_i = f_{NN}(\mathbf{x}_i; \mathbf{w})$. The baseline objective is mean squared error (MSE):

$$\mathcal{L}_{\text{MSE}}(\mathbf{w}) = \frac{1}{B} \sum_{i=1}^{B} (\hat{\sigma}_i - \sigma_i)^2 .$$

MSE is preferred because it is smooth, differentiable everywhere, and strongly penalizes large deviations, which is desirable when the ANN is later used in calibration loops sensitive to local pricing errors. RMSE and MAPE are then used as reporting metrics, following the convention fixed in Chapter 5.

3.4.2. Optimizer And Hyperparameters

In this context, an optimizer is the numerical rule that updates the network weights $\mathbf{w}$ in order to reduce the loss. In a full-gradient method, the update direction would be computed from all training samples at each step. For a dataset of this size, this is unnecessarily expensive, so the practical alternative is to estimate the gradient from mini-batches. This gives a stochastic gradient: it is cheaper to compute, but it introduces sampling noise because each batch only approximates the full empirical gradient.

Adam belongs to the family of stochastic gradient methods, but it modifies plain stochastic gradient descent by using adaptive moments. More precisely, it keeps exponential moving averages of the mini-batch gradient and of its squared values, and uses them to scale the update of each coordinate. The learning rate fixes the base step size before this adaptive rescaling, while the optimizer weight decay controls whether the update also includes an L^2-type shrinkage of the weights. The values used in the finally retained

run are reported in Table 3.3: the Adam learning rate starts at 10^{-3}, and weight decay is set to 0.0, so no additional regularization is imposed through Adam itself. This is useful in our supervised ANN setting because the dataset is large, the loss is non-convex, and the different input directions may produce gradients with different magnitudes.

L-BFGS follows a different logic. It is a limited-memory method that uses recent parameter and gradient differences to build a compact approximation of the local curvature of the loss [27, Ch. 7]. In practical terms, it tries to use more information than the current gradient alone, which can help when the parameters are already close to a good region. The trade-off is computational: each L-BFGS step is heavier and requires reevaluating the loss and its gradient through a closure. For this reason, in this project L-BFGS is mainly tested as a refinement stage after an Adam phase, not as the default training rule for the full ANN run.

The optimizer experiments followed this logic. First, Adam-only runs were used to establish stable baselines under the Liu-style architecture, varying activation, learning-rate schedule, weight decay, and early-stopping policy. Then, mixed schedules were tested by switching from Adam to L-BFGS after an initial stochastic phase. The motivation for these mixed runs was reasonable: Adam can move efficiently through the large dataset, while L-BFGS might improve the final local fit. The retained mixed candidate did not improve the held-out error profile of the selected Adam–`tanh` run, so L-BFGS remains part of the experimental path but not part of the final reference surrogate. The quantitative comparison is reported with the ANN benchmark results in Table 5.1.

Under this criterion, the selected reference ANN is the Liu-style architecture trained with Adam and `tanh` hidden activations. It preserves the reference width and depth, improves on the earlier ReLU baseline in the retained benchmark comparison, and is the ANN used in the final CaNN pipeline. The complete configuration of this selected run is summarized after introducing the regularization and callback controls.

3.4.3. Regularization And Callbacks

Regularization in the selected run is intentionally light: dropout is disabled ($p = 0$), batch normalization is disabled, and Adam weight decay is set to zero. In addition, feature/target normalization is turned off in the Liu-style reference setup. This design keeps the training protocol close to the reference architecture and isolates the effect of the selected hidden activation, optimizer scheduling, and data quality controls.

A learning-rate scheduler is a callback that changes the optimizer step size during training according to a prescribed rule. In the selected run we use StepLR, which reduces Adam's learning rate by a factor γ after a fixed number of epochs. Early stopping follows a different purpose: it monitors validation performance and interrupts training when the selected validation criterion stops improving over a given patience window. In our final run this mechanism is available in the training pipeline but disabled, so model selection

relies on the stored *best* and *last* checkpoints rather than on an automatic stopping rule.

Callback management therefore includes: (i) checkpointing of both *best* and *last* models, (ii) StepLR scheduling for Adam, and (iii) an early-stopping module available in the pipeline but disabled in the baseline experiments. When enabled, early stopping supports both standard validation loss and trimmed validation loss monitors.

From an experimental standpoint, this callback configuration reduces the risk of selecting a late degraded checkpoint without over-constraining the optimization path. The scheduler controls the Adam step size in the selected run, while the Adam-to-L-BFGS transition remains available for mixed experiments. Generalization is then assessed empirically on held-out validation and test splits rather than enforced through strong explicit regularizers.

TABLE 3.3. HYPERPARAMETERS AND TRAINING CONTROLS OF THE SELECTED ANN RUN

Hyperparameter	**Value**
Training mode	Adam
Hidden activation	`tanh`
Seed	42
Epochs	8000
Train batch size	1024
Validation batch size	All validation samples
Loss	MSE
Adam learning rate	10^{-3}
Adam weight decay	0.0
LR scheduler	StepLR, step size = 500, $\gamma = 0.5$
Early stopping	Disabled

Source: Own elaboration from run configuration copies

3.5. Neural Calibration As An Inverse Problem

The supervised ANN pricer constructed in the previous sections approximates the forward Heston map from parameters and contract descriptors to implied volatility. Neural calibration uses the same idea in the opposite direction: given a finite implied-volatility surface, infer the Heston parameter vector

$$\theta = (\rho, \kappa, \gamma, \bar{v}, v_0).$$

Here the parameter meanings are those fixed in Table 2.1, with the same dataset notation convention for $\bar{v}$ and v_0 described in Section 3.2. The calibration workflow follows the neural-calibration idea of Liu et al. [6]: the trained ANN pricer is frozen and used as a fast evaluator inside a calibration loop.

The purpose is not to introduce a new calibration algorithm. The contribution in this thesis is the integration of this calibration step into the ANN–PINN workflow and the diagnostic reading of its output. For that reason, this section keeps only the setup needed to interpret the benchmark in Chapter 5.

3.5.1. Calibration Setup

The trained ANN surrogate approximates the forward map

$$f_{NN} : (\rho, \kappa, \gamma, \bar{v}, v_0, m, \tau, r) \mapsto \sigma_{\mathrm{IV}},$$

where m, τ, r, and σ_{IV} keep the notation fixed in Chapter 2. In calibration, the quote descriptors are fixed and only θ is varied. For each candidate θ, the network is evaluated on the quote set

$$\mathcal{Q} = \{(m_j, \tau_j, r_j, \sigma_j^{\mathrm{mkt}}, w_j)\}_{j=1}^{N_q},$$

where $w_j \geq 0$ is an optional quote weight. The selected benchmark uses $N_q = 35$ quotes on a complete 5×7 moneyness–maturity grid with constant $r = 0.03$; the exact nodes are reported in Section 5.3.

For quote j, the predicted implied volatility and residual are

$$\hat{\sigma}_j(\theta) = f_{NN}(\rho, \kappa, \gamma, \bar{v}, v_0, m_j, \tau_j, r_j), \qquad e_j(\theta) = \hat{\sigma}_j(\theta) - \sigma_j^{\mathrm{mkt}}.$$

Calibration is therefore performed in implied-volatility space, matching the ANN training target and the market convention used for volatility smiles.

The objective minimized in the calibration loop is

$$J(\theta) = \frac{1}{\sum_{j=1}^{N_q} w_j} \sum_{j=1}^{N_q} w_j e_j(\theta)^2 + \lambda R(\theta), \tag{3.1}$$

where $R(\theta)$ is an optional regularization penalty. The calibrated vector is

$$\theta^\star = \arg\min_{\theta \in \mathcal{B}} J(\theta), \tag{3.2}$$

with $\mathcal{B}$ the admissible Heston parameter box. In this thesis, the bounds are aligned with the ANN training domain: $\rho \in [-0.9, 0]$, $\kappa \in [0, 3]$, $\gamma \in [0.01, 0.8]$, $\bar{v} \in [0.01, 0.5]$, and $v_0 \in [0.05, 0.5]$. This restriction keeps calibration inside the intended interpolation regime of the forward surrogate.

3.5.2. Search And Stability Checks

The inverse residual surface is not assumed to be convex, so the reference calibration uses Differential Evolution followed by local polishing. This is a pragmatic choice: the method is robust to initialization, does not require gradients of the neural objective, and can evaluate a population of candidates through batched ANN inference.

The calibration configuration used in the benchmark is fixed in advance and kept inside the interpolation domain of the ANN. It uses the `Liu_like_tanh_1M_v01` checkpoint, the 35-quote grid described above, no regularization in the reference objective ($\lambda = 0$), and the same parameter bounds as the ANN training domain. Differential Evolution is run with maximum iteration count 1000, population multiplier 20, tolerance 10^{-4}, mutation range $(0.5, 1.0)$, recombination 0.7, and final polishing enabled. Regularized variants are reserved for the stability analysis reported in Chapter 5.

The selected solution is assessed at two levels. First, quote-level residuals check whether the implied-volatility surface is reproduced. Second, parameter diagnostics check whether the selected optimum is locally stable. Around a low-residual optimum, the residual data term can be read through the Gauss–Newton approximation

$$H(\theta^\star) \approx 2J_e(\theta^\star)^\top W J_e(\theta^\star),$$

where $J_e(\theta^\star)$ is the Jacobian of quote residuals with respect to θ, and W is the diagonal matrix of quote weights. Large curvature indicates directions where the quote surface reacts strongly to parameter changes; small curvature indicates flatter, less identified directions.

These diagnostics are not interpreted here as results; they define the checks used later to decide whether a calibration is both low-residual and locally stable. The numerical recovery, residual metrics, and parameter-error tables are reported in Chapter 5.

4. PHYSICS-INFORMED NEURAL PRICING UNDER THE HESTON MODEL

This chapter reformulates vanilla option pricing under the Heston model as a PDE-constrained learning problem. In contrast to the supervised ANN surrogate of Chapter 3, the objective is no longer to emulate precomputed numerical labels, but to approximate directly the option-price function through an unsupervised Physics-Informed Neural Network (PINN). The methodology follows the general PINN philosophy and is inspired by the Heston-specific formulation proposed in [10], while adapting the state representation and notation to the conventions used in the present work. Recent physics-informed operator-learning evidence for Heston points to a limitation that is central for this chapter: pricing accuracy alone may coexist with noisy autodiff Delta and Gamma near the at-the-money region [12]. For this reason, the PINN block is evaluated not only as a pricer, but also as a differentiable sensitivity engine.

4.1. Problem formulation

In this chapter, European option pricing under Heston is posed as the approximation of the solution of the pricing PDE rather than as a supervised regression problem. The neural network outputs the option price itself, and the training objective is to enforce that this output satisfies the Heston valuation equation together with its terminal and boundary conditions. Consequently, the core information driving the optimization is not a set of external labels, but the analytical structure of the model under the risk-neutral measure.

To avoid a notational clash between the price function and the variance state, the option value will be denoted by u, while the instantaneous variance state will be denoted by v. This distinction is important because the Heston model contains both a variance process and a pricing function, and using the same symbol for both would make the PDE formulation unnecessarily confusing. The unknown function approximated by the PINN is therefore written as

$$u_\phi(\tau, m, v, \Theta, r),$$

where ϕ denotes the trainable weights of the network, $m = S/K$ is moneyness, $\tau = T - t$ is time-to-maturity, v is the current instantaneous variance, and Θ collects the structural Heston parameters.

More precisely, the calibrated tuple obtained in the previous chapter is

$$\Theta^* = (\rho, \kappa, \gamma, \bar{v}, v_0).$$

However, the role of v_0 must be distinguished from that of the other parameters. The quantities ρ, κ, γ and $\bar{v}$ define the operator of the Heston PDE, whereas v_0 is the initial condition of the variance process. In the PDE formulation, the state variable is the

generic variance level v, not the fixed initial value v_0. Therefore, the parametric PINN is constructed to learn a family of Heston pricing solutions indexed by the structural parameters $(\rho, \kappa, \gamma, \bar{v}, r)$, while the current variance v_0 is used only when evaluating the trained network at the present market state. The calibrated market valuation is thus recovered through

$$u_\phi(\tau, m, v_0, \rho^*, \kappa^*, \gamma^*, \bar{v}^*, r).$$

This point is conceptually important: during training, the PINN learns the solution over a domain in $(\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r)$, whereas at inference time the current market valuation is recovered by fixing the variance coordinate to the calibrated value v_0^* and the structural parameters to the calibrated tuple Θ^*.

The network is trained on collocation points sampled in the interior and on the relevant boundaries of the pricing domain. These points are not generated by simulating full paths of the Heston variance process. Instead, they are drawn directly from a bounded domain of admissible states, for instance using Latin Hypercube Sampling (LHS), and used to evaluate the PDE residual. In that sense, the method is unsupervised: the learning signal comes from the Heston structure itself, not from externally computed price labels. This is the main methodological difference with respect to the ANN surrogate developed in Chapter 3.

At a high level, the proposed workflow is the following. First, a parametric PINN is defined to approximate the price surface over a range of maturities, moneyness levels, variance states, and Heston parameters. Second, the PINN is trained by minimizing the residual of the Heston pricing PDE together with terminal and boundary mismatches. Third, once the network is trained, the price of a contract for a calibrated market configuration is obtained by evaluating the network at the corresponding $(\tau, m, v_0^*, \Theta^*, r)$. This preserves the link with the calibration pipeline while replacing label-based supervision with model-consistent unsupervised training. In this sense, the CaNN stage remains essential: it provides the market-specific parameter tuple that is consumed by the parametric PINN at inference time, without requiring a retraining of the pricing network for each new calibration.

4.1.1. PINN Theoretical Framework

A PINN can be viewed as a parametric approximation of the solution of a differential equation. In the present setting, the network u_ϕ is required to satisfy the Heston pricing PDE in the interior of the domain and to match the contractual conditions at expiry and on selected boundaries. If $\mathcal{N}$ denotes the Heston differential operator, the target problem can be summarized abstractly as

$$\mathcal{N}[u] = 0 \quad \text{in the interior domain,} \qquad \mathcal{B}[u] = 0 \quad \text{on the boundaries,}$$

where $\mathcal{B}$ denotes the set of terminal and boundary constraints.

The PINN replaces the unknown analytical solution by a neural approximation u_ϕ and defines a residual function

$$\mathcal{R}_\phi = \mathcal{N}[u_\phi].$$

Training then consists of minimizing a composite loss built from three sources of error: the interior PDE residual, the mismatch with the terminal payoff, and the mismatch with the selected boundary conditions. In a compact notation, the loss takes the form

$$\mathcal{L}_{\text{PINN}}(\phi) = \mathcal{L}_{\text{PDE}}(\phi) + \mathcal{L}_{\text{term}}(\phi) + \mathcal{L}_{\text{bc}}(\phi).$$

This decomposition is standard in the PINN literature and matches the structure adopted in [10].

An essential ingredient of this approach is automatic differentiation. Since the Heston PDE contains first- and second-order derivatives with respect to the state variables, the network output must remain differentiable with respect to its inputs. The required quantities are obtained by differentiating the computational graph of the network rather than by using finite-difference approximations. This is particularly convenient in the present case because the same differentiable surrogate can later be reused for the computation of Greeks.

4.2. Heston Pricing PDE

Starting from the Heston dynamics introduced in Chapter 2 (Equation 2.13), the risk-neutral version used for pricing is

$$dS_t = rS_t\,dt + \sqrt{v_t}\,S_t\,dW_t^{(1)}, \tag{4.1}$$

$$dv_t = \kappa(\bar{v} - v_t)\,dt + \gamma\sqrt{v_t}\,dW_t^{(2)}, \tag{4.2}$$

with instantaneous correlation

$$d\langle W^{(1)}, W^{(2)}\rangle_t = \rho\,dt.$$

Here r is the risk-free rate, κ is the mean-reversion speed of the variance process, $\bar{v}$ is the long-run variance level, γ is the volatility of variance, and ρ controls the correlation between returns and variance shocks. The current variance level at valuation time is denoted by v_0, so that v_t evolves from the initial condition $v(0) = v_0$.

Let $u(t, S, v)$ denote the price of a European vanilla option. By the Feynman-Kac representation, u satisfies the Heston pricing PDE [3]

$$u_t + \frac{1}{2}vS^2u_{SS} + \rho\gamma vS\,u_{Sv} + \frac{1}{2}\gamma^2 vu_{vv} + rS\,u_S + \kappa(\bar{v} - v)u_v - ru = 0. \tag{4.3}$$

This is the continuous-time constraint that will be enforced by the PINN at interior collocation points. In the present chapter, the price function is denoted by u precisely to keep it separate from the variance state v.

For implementation purposes, it is more convenient to work with time-to-maturity $\tau = T - t$ and moneyness $m = S/K$ instead of calendar time t and spot S. Since the synthetic and market pipelines of this project use a fixed strike normalization $K = 1$, the pricing problem can be rewritten in terms of the state variables (τ, m, v). Defining the same pricing function in transformed coordinates, the PDE becomes

$$u_\tau = \frac{1}{2} v m^2 u_{mm} + \rho \gamma v m u_{mv} + \frac{1}{2} \gamma^2 v u_{vv} + r m u_m + \kappa(\bar{v} - v) u_v - r u. \tag{4.4}$$

This is the formulation that will be used to define the PDE residual in the training loss. It remains fully equivalent to the original Heston pricing equation, but it is better aligned with the representation used in the rest of the thesis.

For a European put option with normalized strike $K = 1$, the terminal condition at expiry is

$$u(0, m, v) = (1 - m)^+. \tag{4.5}$$

The payoff does not depend explicitly on the variance state, which means that the terminal condition is imposed uniformly over the v-dimension of the domain. In addition, a natural lower boundary condition is obtained at zero underlying value:

$$u(\tau, 0, v) = e^{-r\tau}. \tag{4.6}$$

For a general strike K, this condition would read $u(\tau, 0, v) = Ke^{-r\tau}$. In a first PINN specification, these two constraints are sufficient to reproduce the structure used in [10]: one term of the loss is attached to the interior PDE residual, one to the terminal payoff, and one to the lower boundary.

The far-field behavior as $m \to \infty$ for a put, namely $u(\tau, m, v) \to 0$, can also be incorporated if needed. However, to keep the initial implementation simple, the first version of the model will explicitly enforce only the terminal condition and the lower boundary, while controlling the remaining behavior through the bounded sampling domain in (τ, m, v) and the interior residual. This choice is consistent with the objective of building a minimal but coherent unsupervised PINN before adding more elaborate constraints.

4.3. Neural Network Architecture

4.3.1. Model Class

The pricing PINN is implemented as a fully connected feed-forward neural network (MLP) that approximates the mapping

$$u_\phi : \mathbb{R}^8 \to \mathbb{R}, \qquad x = (\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r) \mapsto u_\phi(x),$$

where the scalar output is the option price. This architecture is appropriate for the present setting because all inputs are continuous tabular variables and no spatial or sequential inductive bias is required. This is also the standard architectural choice in many PINN

implementations, where a smooth MLP acts as the differentiable trial function for the PDE solution [10], [15].

Compared with Chapter 3, where the ANN surrogate learned implied volatility from supervised labels, the role of the network here is to represent a differentiable pricing function that is constrained by the Heston PDE and boundary conditions. The model class is therefore selected not only for approximation capacity, but also for stable first- and second-order automatic differentiation. This differentiability requirement is central when PDE residuals or derivative losses are part of the objective, as in gradient-enhanced and Sobolev-style PINN variants [11], [16].

4.3.2. Architecture Details

The reference architecture used in the implementation is

$$8 \to 256 \to 256 \to 256 \to 256 \to 1,$$

with `tanh` activation in hidden layers and linear activation in the output layer. In compact notation,

$$h^{(1)} = \varphi(W^{(1)}x + b^{(1)}),\ \ldots,\ h^{(4)} = \varphi(W^{(4)}h^{(3)} + b^{(4)}), \quad u_\phi = W^{(5)}h^{(4)} + b^{(5)},$$

with $\varphi = \tanh$.

Weights are initialized with Xavier uniform initialization and biases are initialized to zero. In the baseline setup, dropout is disabled and no batch-normalization layers are included.

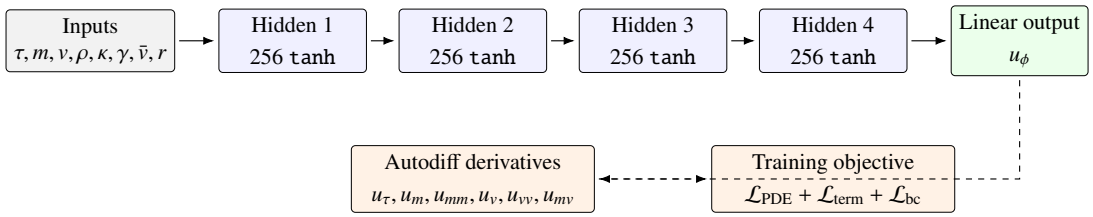


Fig. 4.1. PINN architecture used for Heston pricing. The network maps the eight state and parameter inputs to a scalar option price; automatic differentiation of the same graph provides the derivatives required by the PDE residual and by Greek computation.

TABLE 4.1. PINN ARCHITECTURE USED IN THIS CHAPTER

Hyperparameter	**Value**
Input dimension	8
Hidden layers	4
Hidden width	256 neurons per layer
Hidden activation	`tanh`
Output dimension	1
Output activation	Linear
Weight initialization	Xavier uniform
Bias initialization	Zeros
Dropout	$p = 0.0$
Batch normalization	Disabled

Source: Own implementation (`configs/pinn_model_architecture.yaml`)

4.3.3. Model Inputs and Outputs

The model receives the following ordered input vector:

$$x = (\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r),$$

where (τ, m, v) are state variables, r is the risk-free rate, and $(\rho, \kappa, \gamma, \bar{v})$ are the structural Heston parameters that define the PDE operator.

The network output is a scalar price $u_\phi(x)$, corresponding to the normalized-strike vanilla contract considered in this chapter ($K = 1$). This output choice is consistent with the PDE-constrained formulation: the PINN directly approximates the pricing function rather than an implied-volatility transformation. At inference time, the CaNN calibration block supplies the market-specific values of $(\rho, \kappa, \gamma, \bar{v}, v_0)$; the PINN then evaluates the contract by fixing $v = v_0$ and varying the contract coordinates (τ, m) and the market rate r.

In the current experimental setup, collocation generation is run in a fixed-parameter regime for $(\rho, \kappa, \gamma, \bar{v})$, while (τ, m, v, r) vary across points. This means the model remains formally 8-dimensional, but some coordinates may be constant in a given run due to calibration-conditioned sampling. This is the baseline fixed-Θ benchmark used in the reported experiments; the same input convention leaves the fully parametric variant available as a direct extension.

Input Scaling

Although the pricing problem is formulated in physical variables, the optimization is performed with an affine rescaling of the network inputs. The motivation is numerical: (τ, m, v) and the Heston-related coordinates do not share comparable magnitudes, which

can deteriorate conditioning and gradient balance. Following practical PINN recommendations, centering and scaling are used as a stabilization device; this is consistent with the broader observation that conditioning and gradient-scale imbalance strongly affect PINN training dynamics [13], [14], [15].

In the implemented pipeline, the transformation is applied componentwise to the full PINN input vector through

$$\tilde{x}_d = a_d + b_d x_d, \qquad d = 1, \dots, 8.$$

The coefficients are estimated from the collocation points of the training split:

$$\mu_d = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} x_{i,d}, \qquad \sigma_d = \sqrt{\frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} (x_{i,d} - \mu_d)^2},$$

and then

$$a_d = -\frac{\mu_d}{\sigma_d}, \qquad b_d = \frac{1}{\sigma_d}.$$

To avoid numerical issues on nearly constant coordinates, the implementation uses a small threshold $\varepsilon = 10^{-8}$: if $\sigma_d \le \varepsilon$, that coordinate is left unscaled ($a_d = 0,\ b_d = 1$).

The PDE residual is still defined in physical variables and differentiated by automatic differentiation. In code, the network is evaluated at $\tilde{x}$, while derivatives are taken with respect to the original coordinates x; the chain rule is therefore handled by the computational graph. Consequently, input scaling changes the numerical conditioning of training, but not the financial meaning of the PDE constraint.

4.4. Training Procedure

4.4.1. Loss Function

The PINN is trained by minimizing a composite loss that enforces the pricing PDE in the interior of the domain together with the terminal payoff and the lower boundary condition. Let $u_\phi(\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r)$ denote the output of the neural network. From the transformed Heston PDE introduced above, the residual at an interior point is defined as

$$\mathcal{R}_\phi = u_\tau - \frac{1}{2} v m^2 u_{mm} - \rho\gamma v m u_{mv} - \frac{1}{2}\gamma^2 v u_{vv} - r m u_m - \kappa(\bar{v} - v) u_v + r u, \tag{4.7}$$

where all derivatives are evaluated by automatic differentiation on the computational graph of the network. In the ideal case, the exact pricing function would satisfy $\mathcal{R}_\phi = 0$ for every admissible interior point.

Let $\mathcal{S}_D = \{z_j\}_{j=1}^{N_D}$ denote the set of interior collocation points, with

$$z_j = (\tau_j, m_j, v_j, \rho_j, \kappa_j, \gamma_j, \bar{v}_j, r_j).$$

The interior loss is the mean squared residual

$$\mathcal{L}_{\mathrm{PDE}}(\phi) = \frac{1}{N_D} \sum_{j=1}^{N_D} \mathcal{R}_\phi(z_j)^2. \tag{4.8}$$

On the terminal set $\mathcal{S}_T = \{z_k^T\}_{k=1}^{N_T}$, where $\tau = 0$, the network must match the payoff of the European put. The corresponding error is

$$e_k^T(\phi) = u_\phi(0, m_k, v_k, \rho_k, \kappa_k, \gamma_k, \bar{v}_k, r_k) - (1 - m_k)^+, \tag{4.9}$$

and the terminal loss is defined by

$$\mathcal{L}_{\mathrm{term}}(\phi) = \frac{1}{N_T} \sum_{k=1}^{N_T} \left(e_k^T(\phi)\right)^2. \tag{4.10}$$

On the lower boundary set $\mathcal{S}_{\mathrm{low}} = \{z_l^{\mathrm{low}}\}_{l=1}^{N_{\mathrm{low}}}$, where $m = 0$, the exact put price is the discounted strike. Since the implementation uses the normalized strike $K = 1$, the lower-boundary error is

$$e_l^{\mathrm{low}}(\phi) = u_\phi(\tau_l, 0, v_l, \rho_l, \kappa_l, \gamma_l, \bar{v}_l, r_l) - e^{-r_l \tau_l}, \tag{4.11}$$

and the associated loss is

$$\mathcal{L}_{\mathrm{bc}}(\phi) = \frac{1}{N_{\mathrm{low}}} \sum_{l=1}^{N_{\mathrm{low}}} \left(e_l^{\mathrm{low}}(\phi)\right)^2. \tag{4.12}$$

The total training objective is then written as

$$\mathcal{L}_{\mathrm{PINN}}(\phi) = \mathcal{L}_{\mathrm{PDE}}(\phi) + \mathcal{L}_{\mathrm{term}}(\phi) + \mathcal{L}_{\mathrm{bc}}(\phi). \tag{4.13}$$

For implementation, it is convenient to keep an explicit weighted form,

$$\mathcal{L}_{\mathrm{PINN}}(\phi) = \lambda_{\mathrm{PDE}} \mathcal{L}_{\mathrm{PDE}}(\phi) + \lambda_{\mathrm{term}} \mathcal{L}_{\mathrm{term}}(\phi) + \lambda_{\mathrm{low}} \mathcal{L}_{\mathrm{bc}}(\phi), \tag{4.14}$$

with non-negative coefficients $(\lambda_{\mathrm{PDE}}, \lambda_{\mathrm{term}}, \lambda_{\mathrm{low}})$ that can be tuned if one component dominates the optimization. In the baseline specification of this thesis, all three weights are initialized to one, i.e. $(1, 1, 1)$, following the simplest formulation in [10].

A practical point is that the PDE term requires second-order derivatives with respect to inputs. Therefore, the automatic-differentiation graph must be kept when computing first derivatives, so that mixed and second derivatives such as u_{mv} and u_{vv} can be obtained consistently in the same computational graph. This is the key implementation detail that enables direct optimization of the PDE residual without finite-difference approximations.

4.4.2. Optimizer and Hyperparameters

The optimization follows a two-stage strategy that is common in PINN practice: an initial stochastic phase with Adam, followed by a quasi-Newton refinement with L-BFGS. In this work, this strategy is implemented as a mixed schedule in which Adam is used during the first part of training and L-BFGS in the second part, while a pure Adam or pure L-BFGS run remains available as an ablation.

Since the three loss terms are evaluated on different sample sets, each epoch uses three mini-batches: one from $\mathcal{S}_D$ (interior), one from $\mathcal{S}_T$ (terminal), and one from $\mathcal{S}_{\text{low}}$ (lower boundary). This allows different batch sizes for interior and boundary constraints and keeps the training loop aligned with the composite objective.

The reference hyperparameter block for the first implementation includes: learning rates and optimizer-specific settings, the Adam-to-L-BFGS switching epoch in mixed mode, batch sizes for interior and boundary sets, and the triplet of loss weights $(\lambda_{\text{PDE}}, \lambda_{\text{term}}, \lambda_{\text{low}})$.

For diagnostics, the training run stores epoch-wise train/validation curves for the total weighted loss and for each component separately. In addition, a two-dimensional error map over (m, τ) is produced from validation interior points by averaging $|\mathcal{R}_\phi|$ in bins. This map helps identify whether specific regions of the contract domain remain systematically underfitted even when the global loss decreases.

4.4.3. Sampling strategy

Training requires three different sample sets, each associated with one component of the loss. As in the general PINN methodology, these sets are not built from labeled prices but from points drawn directly in the state-parameter domain. The first set corresponds to the interior of the PDE domain, the second to the terminal boundary at expiry, and the third to the lower boundary at zero underlying value.

The interior domain is defined over bounded intervals for the state variables and for the Heston parameters:

$$\tau \in [0, \tau_{\max}], \qquad m \in [m_{\min}, m_{\max}], \qquad v \in [v_{\min}, v_{\max}],$$

combined with admissible ranges for $(\rho, \kappa, \gamma, \bar{v}, r)$. These parameter intervals should remain consistent with the calibration regime studied in Chapter 3, since the purpose of the parametric PINN is precisely to generalize over the family of Heston configurations that may be produced by the CaNN stage. In particular, the v-dimension is sampled explicitly as a state coordinate of the PDE. This is a crucial point: the PINN is trained on generic variance states v, whereas the calibrated initial variance v_0 is only used later, when evaluating the trained network at the current market state.

To obtain a representative coverage of this multidimensional domain with a controlled number of points, the baseline sampling method will be Latin Hypercube Sam-

pling (LHS). Compared with naive Monte Carlo sampling, LHS provides a more regular coverage of each marginal dimension and reduces the risk of leaving large regions of the domain underexplored. In the present setting, this is particularly useful because the PINN must generalize simultaneously over state variables and model parameters.

For the interior set $\mathcal{S}_D$, each sampled point has the form

$$z_j = (\tau_j, m_j, v_j, \rho_j, \kappa_j, \gamma_j, \bar{v}_j, r_j), \qquad j = 1, \ldots, N_D.$$

These are the collocation points at which the PDE residual is evaluated. For the terminal set $\mathcal{S}_T$, points are sampled in the subdomain

$$\tau = 0, \qquad m \in [m_{\min}, m_{\max}], \qquad v \in [v_{\min}, v_{\max}],$$

again jointly with the Heston parameters. This set is used to impose the payoff condition. For the lower-boundary set $\mathcal{S}_{\text{low}}$, the sampled points satisfy

$$m = 0, \qquad \tau \in [0, \tau_{\max}], \qquad v \in [v_{\min}, v_{\max}],$$

with the same parameter ranges, and are used to enforce the discounted-strike boundary condition.

From an implementation viewpoint, the three sets can be regenerated dynamically during training or precomputed once and reused across epochs. A dynamic resampling strategy may improve coverage of the domain and reduce overfitting to a fixed cloud of collocation points, whereas a static design simplifies reproducibility and debugging. The initial implementation will prioritize simplicity and reproducibility; therefore, the first reference version will rely on fixed sample sets generated before the training loop. Dynamic resampling can later be evaluated as an extension if it leads to more stable convergence.

This sampling strategy preserves the fully unsupervised nature of the method. No prices from COS, FFT, or Monte Carlo are required to build the training objective. Numerical pricing methods remain useful for ex-post validation and benchmarking, but not as a source of labels for fitting the PINN itself.

4.5. Extension: Sobolev-Enhanced Training for Greek Accuracy

The baseline PINN objective in this chapter is designed to recover prices by enforcing the Heston PDE and boundary/terminal conditions. However, accurate prices do not automatically imply accurate first- and second-order sensitivities. In particular, Γ and variance/rate sensitivities may show larger local errors because they depend on curvature properties of the learned surface. This issue is also observed in recent physics-informed Heston operator learning, where autodiff Greeks become noisy even when price errors are small [12]. To address this limitation, a natural extension is to enrich training with derivative-aware terms inspired by Sobolev training [16] and gradient-enhanced PINN

formulations [11]. The formulation below is an application-specific hybrid: it follows Sobolev Training in matching target derivatives, but unlike gPINN it does not penalize gradients of the PDE residual directly.

The extension keeps the original physics objective and adds supervised derivative terms on a dedicated set of Greek-anchor points. Let

$$\mathcal{L}_{\text{base}} = \lambda_{\text{pde}}\mathcal{L}_{\text{pde}} + \lambda_{\text{term}}\mathcal{L}_{\text{term}} + \lambda_{\text{low}}\mathcal{L}_{\text{low}},$$

and let $\mathcal{S}_{\text{g}}$ be a set of anchor points in the same input space

$$x = (\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r).$$

For each $x \in \mathcal{S}_{\text{g}}$, reference sensitivities $\mathcal{G}^{\star}(x)$ are obtained from the semi-analytical Heston characteristic-function map (Section 2.5.2), while $\mathcal{G}_{\phi}(x)$ are computed from autodiff on the PINN output.

To prevent scale dominance across Greeks, we use normalized residuals and split supervision into two blocks:

$$\mathcal{L}_{\text{g},1} = \frac{1}{|\mathcal{S}_{\text{g}}|\,|\mathcal{Q}_1|} \sum_{x\in\mathcal{S}_{\text{g}}} \sum_{q\in\mathcal{Q}_1} \left(\frac{q_{\phi}(x) - q^{\star}(x)}{s_q + \varepsilon} \right)^2, \qquad \mathcal{Q}_1 = \{\Delta, \text{Vega}_v, \Theta, \text{Rho}\}, \tag{4.15}$$

$$\mathcal{L}_{\text{g},2} = \frac{1}{|\mathcal{S}_{\text{g}}|} \sum_{x\in\mathcal{S}_{\text{g}}} \left(\frac{\Gamma_{\phi}(x) - \Gamma^{\star}(x)}{s_{\Gamma} + \varepsilon} \right)^2, \tag{4.16}$$

where s_q are fixed scale factors (e.g., robust quantiles of $|q^{\star}|$ on $\mathcal{S}_{\text{g}}$) and $\varepsilon > 0$ avoids numerical singularities near zero. The separation of $\mathcal{L}_{\text{g},2}$ for Γ gives independent control of curvature supervision, which is typically more unstable than first-order terms.

The prototype objective is then

$$\mathcal{L}_{\text{proto}} = \mathcal{L}_{\text{base}} + \lambda_{\text{g},1}\mathcal{L}_{\text{g},1} + \lambda_{\text{g},2}\mathcal{L}_{\text{g},2}. \tag{4.17}$$

From an optimization viewpoint, this extension introduces additional multi-term imbalance risks, already documented in the PINN literature [13], [14], [15]. Therefore, the intended training strategy is two-stage: first, train the baseline PINN to a stable price solution; second, apply a short fine-tuning phase with gradually increased $(\lambda_{\text{g},1}, \lambda_{\text{g},2})$, while monitoring possible regressions in price/PDE metrics.

A stricter physics-informed alternative would be to differentiate the Heston PDE itself with respect to the state variables and penalize the residuals of the PDEs satisfied by the sensitivities. This is the type of Sobolev or sensitivity-equation route suggested in [12]. It has the advantage of reducing the dependence on external Greek labels, but it also introduces higher-order derivatives and additional loss-balancing difficulties. The experiment in this thesis therefore uses semi-analytical Heston Greeks as derivative targets: this makes the diagnostic controlled and directly comparable with the reference engine, while leaving sensitivity-PDE training as a natural extension.

4.6. Greeks Computation

4.6.1. Definition of Greeks from the PINN Output

Once the PINN has been trained, the network output is a differentiable approximation of the vanilla option price:

$$u_\phi(\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r).$$

Therefore, sensitivities can be obtained directly as partial derivatives of this function, evaluated at a chosen market point. Let

$$x = (\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r).$$

The first-order and second-order derivatives with respect to the input coordinates are denoted by

$$\nabla_x u_\phi(x), \qquad \nabla_x^2 u_\phi(x).$$

From these objects, standard trading Greeks are extracted as follows:

$$\Delta = \frac{\partial u_\phi}{\partial S}, \tag{4.18}$$

$$\Gamma = \frac{\partial^2 u_\phi}{\partial S^2}, \tag{4.19}$$

$$\Theta = \frac{\partial u_\phi}{\partial t} = -\frac{\partial u_\phi}{\partial \tau}, \tag{4.20}$$

$$\text{Rho} = \frac{\partial u_\phi}{\partial r}. \tag{4.21}$$

In this work, the spatial coordinate is moneyness $m = S/K$, not spot directly. Hence a chain-rule conversion is required:

$$\frac{\partial u_\phi}{\partial S} = \frac{\partial u_\phi}{\partial m}\frac{\partial m}{\partial S} = \frac{1}{K}\frac{\partial u_\phi}{\partial m}, \tag{4.22}$$

$$\frac{\partial^2 u_\phi}{\partial S^2} = \frac{1}{K^2}\frac{\partial^2 u_\phi}{\partial m^2}. \tag{4.23}$$

For the normalized-strike setup used in the thesis ($K = 1$), both coordinate systems coincide numerically.

Since the PINN state includes instantaneous variance v, a natural model-consistent volatility sensitivity is

$$\text{Vega}_v := \frac{\partial u_\phi}{\partial v}.$$

If needed, this can be transformed to a sensitivity with respect to volatility level $\sigma_v = \sqrt{v}$:

$$\frac{\partial u_\phi}{\partial \sigma_v} = 2\sqrt{v}\,\frac{\partial u_\phi}{\partial v}.$$

In addition, the parametric PINN provides direct structural sensitivities

$$\frac{\partial u_\phi}{\partial \rho}, \quad \frac{\partial u_\phi}{\partial \kappa}, \quad \frac{\partial u_\phi}{\partial \gamma}, \quad \frac{\partial u_\phi}{\partial \bar{v}},$$

which are useful for risk decomposition and calibration diagnostics beyond classical desk Greeks.

4.6.2. Automatic Differentiation of the PINN

The derivative computation is based on automatic differentiation over the computational graph of the trained network. For a single input point x, we compute:

$$u_\phi(x), \qquad J(x) = \nabla_x u_\phi(x), \qquad H(x) = \nabla_x^2 u_\phi(x).$$

Because the model is differentiable end-to-end, these quantities are exact derivatives of the neural surrogate (up to floating-point precision), not finite-difference approximations.

Compared with bump-and-reprice finite differences, this approach has three practical advantages in the PINN context:

- it avoids step-size tuning and the associated truncation/cancellation trade-off;
- it provides first- and second-order sensitivities in a single coherent graph;
- it scales efficiently to batched inputs through vectorized Jacobian/Hessian evaluation.

From a financial perspective, the interpretation remains standard: each Greek is the local slope or curvature of the price surface produced by the PINN around the evaluation point. The quality of those sensitivities therefore depends on both the local smoothness of the trained network and the fidelity of the learned price function in that region. This closes the link with the Heston dynamics introduced in Chapter 2: the PINN price surface is trained through the PDE implied by Equation 2.13, and the Greeks are then derivatives of that model-consistent learned surface.

5. NEURAL NETWORK PERFORMANCE AND BENCHMARKING

5.1. Experimental Setup And Evaluation Protocol

This chapter compares the three model blocks developed in the thesis under a common evaluation protocol: ANN pricing surrogate, calibration network workflow, and PINN-based pricing and Greeks computation.

Unless explicitly stated, all reported metrics are computed on fixed evaluation sets with identical feature conventions, consistent preprocessing, and the same numerical references used in previous chapters.

5.1.1. Primary Error Metrics

To keep the empirical comparison coherent, the chapter uses three primary error metrics throughout: MSE, RMSE, and MAPE. Let z_i^{ref} be the reference value and $\hat{z}_i$ the model value at evaluation point i, for $i = 1, \ldots, n$. The metrics are

$$\begin{aligned} \mathrm{MSE} &= \frac{1}{n}\sum_{i=1}^{n}\left(\hat{z}_i - z_i^{\mathrm{ref}}\right)^2, \\ \mathrm{RMSE} &= \sqrt{\mathrm{MSE}}, \\ \mathrm{MAPE} &= 100\,\frac{1}{n}\sum_{i=1}^{n}\frac{\left|\hat{z}_i - z_i^{\mathrm{ref}}\right|}{\max\left(\left|z_i^{\mathrm{ref}}\right|, \varepsilon_{\mathrm{MAPE}}\right)}. \end{aligned} \tag{5.1}$$

MSE is aligned with the training objective used in the supervised components, RMSE restores the error to the natural output units, and MAPE provides a scale-relative percentage reading. The denominator floor $\varepsilon_{\mathrm{MAPE}}$ avoids unstable ratios near zero. Tail quantiles and maximum errors are used only as auxiliary diagnostics when discussing distributional behavior, not as headline metrics.

The minimal pre-benchmark validation protocol combines three checks: convergence with respect to (N, L) in the COS expansion, put-call parity residual control across strikes, and Black–Scholes consistency in near-constant-variance regimes.

These checks are used as a guardrail before comparing models, so that downstream differences can be attributed to model behavior rather than to numerical artifacts. For consistency across Sections 5.2–5.5, each block is analyzed with the same reading logic: global summary metrics, localized error diagnostics over relevant domains, and runtime indicators.

The protocol is fixed as follows. Randomness is controlled with a common seed (42) in data generation and training configurations; in calibration, the Differential Evolution

seed follows the same base value unless explicitly overridden. The Heston domain is shared across experiments, with $\rho \in [-0.9, 0]$, $\kappa \in [0, 3]$, $\gamma \in [0.01, 0.8]$, $\bar{v} \in [0.01, 0.5]$, $v_0 \in [0.05, 0.5]$, and contract coordinates $m \in [0.6, 1.4]$, $\tau \in [0.05, 3.0]$ for ANN/CaNN benchmarks. Reference numerics are also fixed: COS pricing with $N = 1500$, $L = 50$, and fallback $L = 80$ when required by stability checks, plus Brent IV inversion with tol $= 10^{-8}$ and bounded search interval.

Reported metrics are always computed on fixed held-out sets, with ANN split policy fixed to 0.8/0.1/0.1 (train/validation/test), and values reported in decimal units (IV for ANN/CaNN, price for PINN). Finally, each model block reports equivalent evidence layers (global table, regional/spatial diagnostic, and convergence or runtime indicator), so cross-model comparisons remain methodologically aligned.

Methodological details are kept in the corresponding method chapters: Chapter 3 for the supervised ANN/CaNN block and Chapter 4 for PINN. This chapter focuses on consolidated benchmark outcomes under the protocol above.

5.2. ANN Pricer Performance

This section reports the performance of the supervised ANN pricer introduced in Chapter 3. The objective is to assess mean implied-volatility accuracy and spatial stability over (m, τ), using a fully specified evaluation protocol that can be reproduced without implementation-specific references.

The ANN analyzed here corresponds to the selected tanh configuration from Chapter 3. The benchmark uses fixed weights (no retraining in Chapter 5) and fixed train/-validation/test splits of sizes $8.0 \times 10^5/1.0 \times 10^5/1.0 \times 10^5$. Pointwise error is defined as

$$e = \hat{\sigma}_{\mathrm{IV}} - \sigma_{\mathrm{IV}}^{\mathrm{ref}},$$

where $\hat{\sigma}_{\mathrm{IV}}$ is the ANN-predicted implied volatility and $\sigma_{\mathrm{IV}}^{\mathrm{ref}}$ is the reference implied volatility obtained from the Heston-COS/Brent pipeline. The error e is reported in decimal implied-volatility units.

Global quality is summarized with MSE, RMSE, and MAPE. Regional diagnostics use fixed bins over m and τ, and extrapolation behavior is analyzed through sensitivity maps with explicit interpolation limits (Figure 5.1).

Before reporting the final split-wise metrics, Table 5.1 summarizes the retained candidates used to select the reference ANN. The table uses descriptive names rather than internal run identifiers: all candidates follow the same Liu-style depth and width, while the activation and update schedule are the relevant differences.

TABLE 5.1. OPTIMIZER-SELECTION RESULTS FOR RETAINED ANN CANDIDATES. ALL ROWS USE THE SAME TRAIN/VALIDATION/TEST SPLIT POLICY AND LIU-STYLE NETWORK SIZE.

Candidate	**Activation**	**Update schedule**	**Val MSE**	**Test RMSE**	**Test MAPE**
Selected tanh ANN	`tanh`	Adam	4.00×10^{-6}	1.86×10^{-3}	15.88%
ReLU baseline	ReLU	Adam	4.65×10^{-6}	3.03×10^{-3}	18.33%
Adam–L-BFGS refinement	ReLU	Adam → L-BFGS	4.55×10^{-6}	3.07×10^{-3}	27.17%

Source: Own elaboration from stored ANN evaluation summaries

5.2.1. Global Error Metrics

Table 5.2 reports split-wise global errors for the selected ANN baseline.

TABLE 5.2. GLOBAL ANN IMPLIED-VOLATILITY ERRORS (SELECTED TANH CONFIGURATION).

Split	**n**	**MSE**	**RMSE**	**MAPE (%)**
Train	8.0×10^5	6.99×10^{-8}	2.64×10^{-4}	2.21
Validation	1.0×10^5	4.00×10^{-6}	2.00×10^{-3}	18.12
Test	1.0×10^5	3.47×10^{-6}	1.86×10^{-3}	15.88

Source: Own elaboration

The test RMSE remains in the 10^{-3} range in absolute IV units, with a lower test MSE/RMSE than the earlier ReLU reference run. MAPE is larger than the raw IV error scale suggests because relative errors are amplified in low-IV regions; therefore it is read jointly with the spatial diagnostics below rather than as an isolated scalar.

Figure 5.1 provides a panel-wise diagnostic. Each row corresponds to one parameter plane: top $(\gamma, \bar{v})$ (vol-of-vol versus long-run variance), middle (ρ, γ) (correlation versus vol-of-vol), and bottom $(\kappa, \bar{v})$ (mean reversion versus long-run variance). The left column shows ANN IV maps with white isoclines, while the right column shows $\log|\,\mathrm{NN} - \mathrm{Heston}\,|$ on the same grids, using lighter tones for lower errors and darker tones for larger errors. The dashed black lines delimit interpolation limits; regions beyond those limits are extrapolation zones where the network was not trained.

The most stable behavior is observed in the interior of each plane, where isoclines are smooth and error colors remain comparatively light. Error increases near boundary segments and especially outside the dashed limits, with the strongest deterioration

concentrated in outer corners of the tested planes. This is consistent with the regional-error reading: the surrogate remains accurate over the interpolation core, while low-moneyness/short-maturity-like boundary regimes are the most fragile. In practical terms, Figure 5.1 supports using the ANN as a robust interpolation surrogate and treating extrapolated regions as lower-confidence zones.

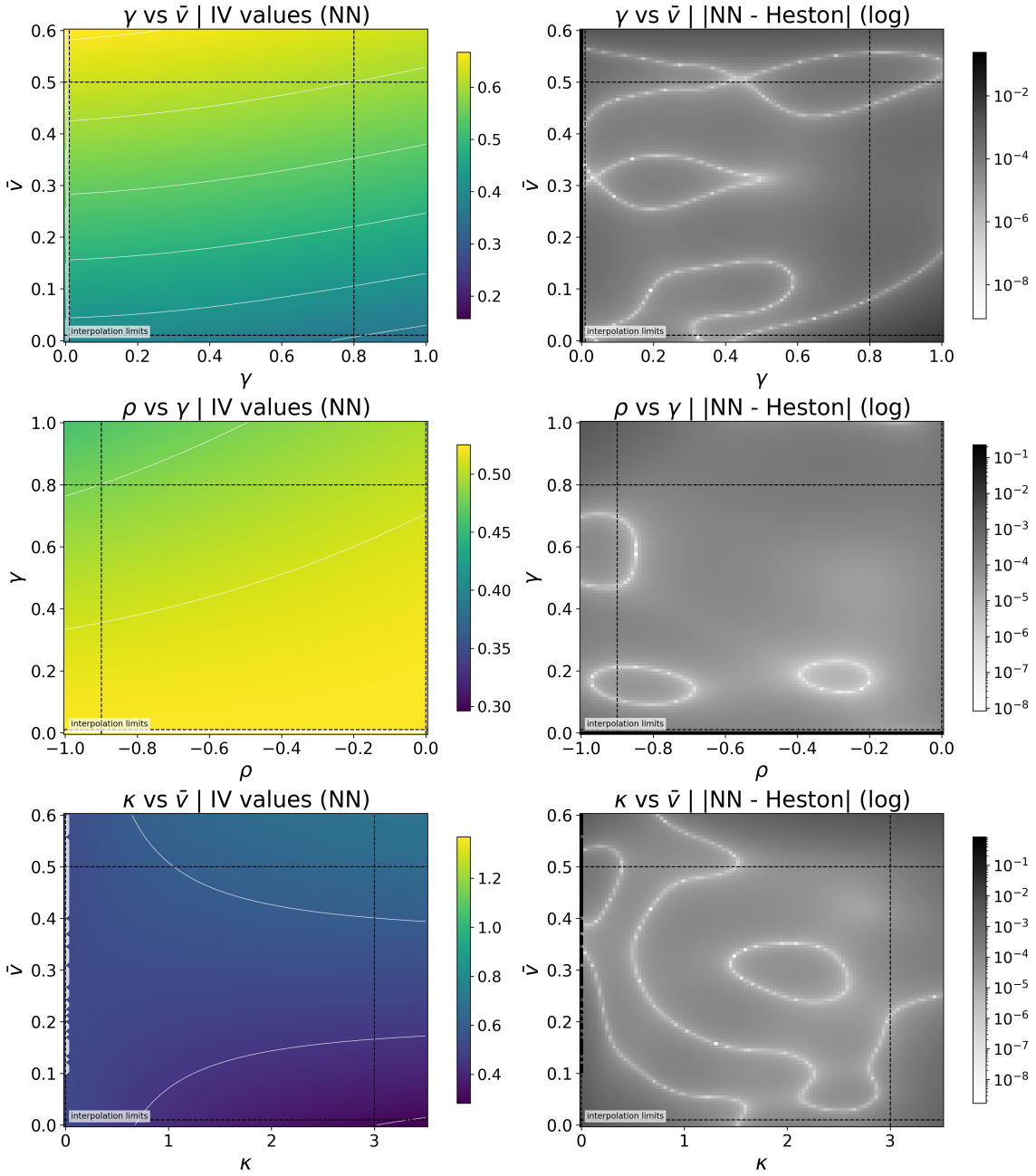


Fig. 5.1. Sensitivity grid for the ANN pricer. Top row: $(\gamma, \bar{v})$, middle row: (ρ, γ), bottom row: $(\kappa, \bar{v})$. Left panels report IV-value maps with white isoclines; right panels report $\log|\mathrm{NN}-\mathrm{Heston}|$, where lighter tones indicate lower error and darker tones higher error. Dashed black lines indicate interpolation limits.

5.2.2. Regional Error Profile

Regional diagnostics identify where approximation error concentrates. On the test split, the largest MSE values are concentrated at low moneyness and short maturity: the bin $m \in [0.6, 0.8]$ accounts for roughly one quarter of test points, while $\tau \in [0.05, 0.25]$ accounts for roughly 6.8%. Interior and longer-horizon bins show a clear error drop, so the pattern is structural rather than driven by isolated rare points.

The same pattern appears in the regional MSE curves by moneyness and maturity bins. In both validation and test curves, the first bin is clearly the most difficult, while errors drop sharply and remain much lower in interior bins. This is consistent with boundary/extrapolation effects and with the smoother behavior observed around central bins.

5.3. Calibration Network Performance

This section reports the benchmark outcomes of the CaNN workflow. The compact calibration setup and diagnostic reading are introduced in Section 3.5; here we focus on final metrics and on the selection of the reference calibration run used in the rest of the chapter.

5.3.1. Benchmark Snapshot

The calibration benchmark in this section follows the inverse-problem construction of Section 3.5.1: it uses a fixed ANN surrogate (from Section 5.2) and one fixed quote universe of 35 contracts (Table 5.3). The calibrated vector is $(\rho, \kappa, \gamma, \bar{v}, v_0)$, optimized with Differential Evolution plus polish under the same bounds and controls defined in Sections 3.5.1–3.5.2.

TABLE 5.3. QUOTE UNIVERSE USED IN THE CANN BENCHMARK.

Item	Value
Number of quotes	35
Grid structure	Complete 5×7 grid over (m, τ)
Moneyness nodes m	$\{0.85, 0.925, 1.00, 1.075, 1.15\}$
Maturity nodes τ	$\{0.50, 0.75, 1.00, 1.25, 1.50, 1.75, 2.00\}$
Interest rate r	0.03 (constant across quotes)

Source: Own elaboration

5.3.2. Fit Quality And Convergence

The final Liu-like calibration run uses the selected ANN width and depth from Chapter 3, with `tanh` hidden activations and Xavier uniform initialization. This tanh configuration

supersedes the earlier ReLU calibration in the final benchmark because it gives a lower quote residual and a substantially better recovery of the synthetic truth parameters.

To characterize residual magnitudes directly in IV units, Table 5.4 summarizes the selected run with the same primary metric set used throughout the chapter.

TABLE 5.4. CANN QUOTE-LEVEL RESIDUAL METRICS (SELECTED TANH RUN).

n	MSE	RMSE	MAPE (%)
35	7.43×10^{-10}	2.73×10^{-5}	0.0048

Source: Own elaboration

Residual concentration is very small at quote level. The largest absolute residual in the selected tanh run is 6.79×10^{-5} in IV units, and the signed residuals remain balanced around zero over the (m, τ) grid. The resulting quote fit is therefore one order of magnitude tighter than the earlier ReLU calibration used during the intermediate experiments.

5.3.3. Parameter Recovery Profile

Since this quote set includes sidecar synthetic truth, parameter-level errors can be reported directly. Table 5.5 keeps the comparison focused on the final selected run and the Liu et al. reference values.

TABLE 5.5. SELECTED TANH CANN RUN VERSUS LIU ET AL. (ABSOLUTE PARAMETER ERRORS).

Metric	This work (selected tanh run)	Liu et al. [6]
Weighted MSE	7.43×10^{-10}	$\sim 10^{-8}$
$\lvert\rho - \rho^\star\rvert$	8.05×10^{-3}	4.84×10^{-2}
$\lvert\kappa - \kappa^\star\rvert$	8.48×10^{-4}	4.88×10^{-2}
$\lvert\gamma - \gamma^\star\rvert$	6.57×10^{-3}	3.28×10^{-2}
$\lvert\bar{v} - \bar{v}^\star\rvert$	1.02×10^{-4}	4.54×10^{-3}
$\lvert v_0 - v_0^\star\rvert$	1.83×10^{-4}	4.39×10^{-4}

Source: Own elaboration based on selected run outputs and [6].

In Table 5.5, *Weighted MSE* denotes the weighted mean-squared IV residual used as calibration objective. Under the replicated 35-quote Liu grid, the selected tanh run improves every reported parameter error relative to the Liu et al. values. The largest absolute deviations still occur in ρ and γ, but their scale is reduced from 10^{-2}–10^{-1} to the 10^{-3}–10^{-2} range. This is the strongest calibration result obtained in the thesis and is the CaNN result used in the final interpretation.

5.3.4. Local Smoothness And Curvature Study

To characterize local identifiability around the calibrated optimum, Jacobian and Hessian diagnostics were computed at $\theta^\star$ for the selected tanh run.

TABLE 5.6. LOCAL-CURVATURE DIAGNOSTICS AT THE CALIBRATED SOLUTION $\theta^\star$ (SELECTED TANH RUN).

$\|\nabla J(\theta^\star)\|_2$	$\lambda_{\min}(H)$	$\lambda_{\max}(H)$	$\kappa_{\text{abs}}(H)$
8.75×10^{-6}	4.69×10^{-5}	4.18×10^{1}	8.89×10^{5}

Source: Own elaboration

The selected solution satisfies near-stationarity ($\|\nabla J(\theta^\star)\|_2 \ll 10^{-4}$). The Hessian is symmetric, and the reported extreme eigenvalues are both non-negative in this run, consistent with a locally convex basin around the selected optimum. At the same time, $\kappa_{\text{abs}}(H) \approx 8.9 \times 10^5$ indicates pronounced anisotropy (flat and stiff directions coexist). The strongest cross-parameter coupling appears between $\bar{v}$ and v_0 (off-diagonal entry ≈ 18.47), followed by (γ, v_0) and $(\gamma, \bar{v})$, which is coherent with the observed recovery pattern.

The Jacobian components at $\theta^\star$ remain close to zero (between 10^{-10} and 10^{-6}), reinforcing first-order convergence. The largest entries are associated with $\bar{v}$ and v_0, but their magnitude is still very small, so there is no dominant residual slope left at the optimum.

The Hessian is the key second-order diagnostic. It has a block structure centered on $(\bar{v}, v_0)$: both diagonal entries are the largest (high local curvature), and their off-diagonal term is also large, which indicates strong local coupling between long-run variance and initial variance. By contrast, the κ-related diagonal curvature is much smaller, so perturbations in that direction are less strongly penalized around $\theta^\star$. This mixed pattern (stiff variance block plus flatter directions in other coordinates) explains why fit residuals can be very small while some parameters, especially ρ and γ, remain harder to pin down with the same precision as $(\bar{v}, v_0)$.

The results above define the selected CaNN benchmark used in the rest of this chapter.

5.4. PINN Pricing Performance

This section reports the baseline PINN pricing quality before introducing Sobolev terms in the training objective. The goal is to establish a reproducible reference for price-level accuracy and runtime against the COS/Heston engine.

The detailed reproducible setup (input convention, standardization, COS parameters, filtering, and arithmetic context) is moved to Appendix C. In short, the evaluated model is the baseline parametric PINN without Sobolev terms, benchmarked against the Heston-COS reference on 9.996×10^3 valid comparisons after numerical filtering.

5.4.1. Global Pricing Metrics

The compact global summary is embedded in Figure 5.2: MSE = 3.1621×10^{-5}, RMSE = 5.6233×10^{-3}, and MAPE = 16.01%. These results are obtained with the baseline PINN only (without Sobolev augmentation). In runtime terms, the PINN evaluates the full benchmark in 0.0203 s ($\approx 4.91 \times 10^5$ points/s), while COS requires 114.47 s (≈ 87.36 points/s), i.e., an empirical throughput ratio of approximately 5.63×10^3.

5.4.2. Spatial And Distributional Diagnostics

The pointwise agreement between PINN and COS prices, the global metric table, the spatial error maps, and the distributional diagnostic are consolidated in Figure 5.2. The scatter panel remains tightly concentrated around the identity line, while a mild negative bias (mean error -1.87×10^{-3}) indicates slight underpricing in the central price range.

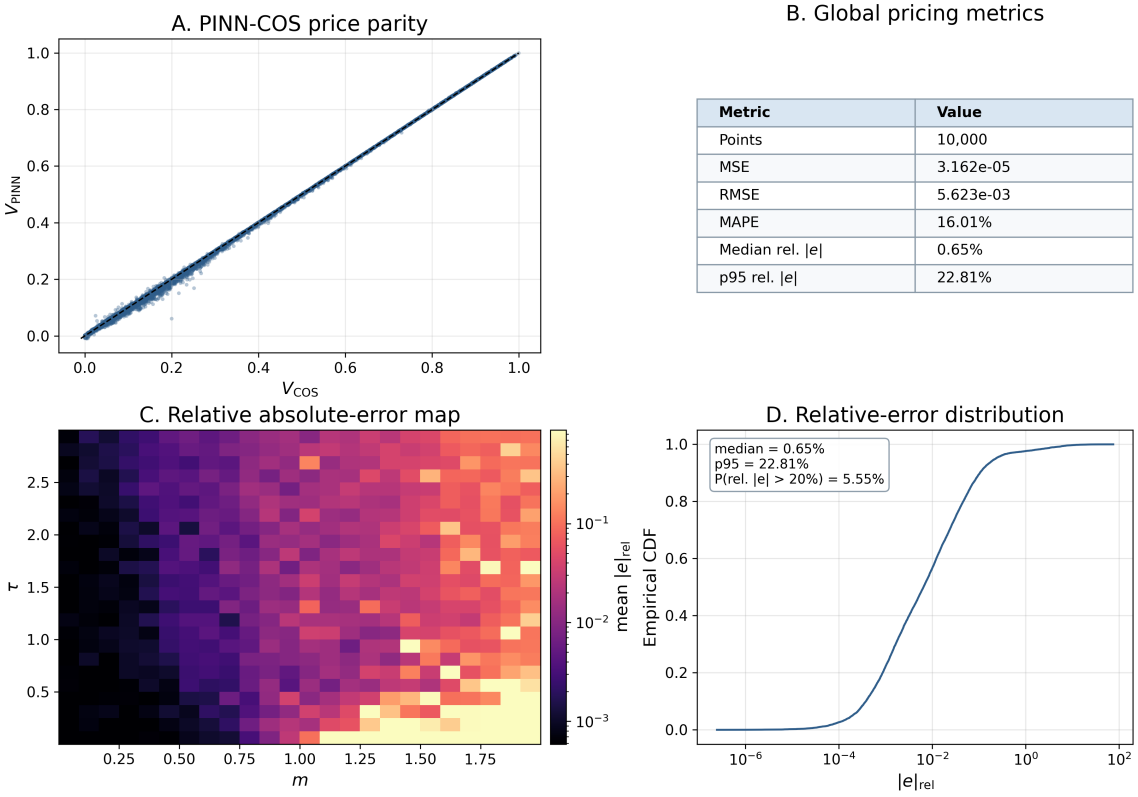


Fig. 5.2. Composite PINN pricing benchmark versus COS reference. Top-left: PINN-COS scatter. Top-right: global MSE/RMSE/MAPE and relative-error summary. Bottom-left: mean relative absolute-error map over (m, τ). Bottom-right: empirical CDF of relative absolute pricing errors.

The spatial panel shows that relative error is not homogeneous over (m, τ): the largest relative deviations are concentrated in low-reference-price regions, especially towards high moneyness and shorter maturities, whereas the central region is markedly more stable. These values must be interpreted jointly with the absolute-unit metrics (MSE and

RMSE): when the COS reference price is close to zero, relative error inflates, so local high percentages do not necessarily correspond to large monetary mispricing. Distributionally, the median relative absolute error is 0.65%, the p_{95} relative absolute error is 22.81%, and 5.55% of points exceed 20% relative error.

5.5. PINN Greeks Performance

This section evaluates autodiff-based Greek quality for the baseline PINN and quantifies the effect of Sobolev augmentation through a controlled ablation.

5.5.1. Ablation Design And Reproducible Protocol

The ablation isolates the impact of Jacobian-informed Sobolev terms on top of the baseline PINN objective, comparing:

- **Baseline PINN**: PDE + terminal + lower-boundary losses only.
- **Sobolev-augmented PINN**: baseline loss plus derivative supervision terms $\mathcal{L}_{g,1}$ and $\mathcal{L}_{g,2}$ as defined in Chapter 4.

To keep the comparison interpretable, both variants are benchmarked on the same (m, τ) grid, with identical fixed parameters, the same Heston characteristic-function Greek engine, and the same numerical integration settings.

TABLE 5.7. REPRODUCIBLE PROTOCOL FOR THE GREEK ABLATION BENCHMARK.

Item	Value
Compared variants	Baseline PINN vs Sobolev-augmented PINN
Architecture and features	Fixed across variants (same parametric PINN input convention)
Evaluation grid	$m \in [0.8, 1.2]$ (161 points), $\tau \in [0.05, 2.0]$ (161 points)
Total evaluation size	$n = 2.5921 \times 10^4$ points per Greek
Fixed model parameters	$v = 0.04,\ \rho = -0.7,\ \kappa = 2.0,\ \gamma = 0.3,\ \bar{v} = 0.04,\ r = 0.01$
Reference Greeks	Semi-analytical Heston characteristic-function Greeks
Characteristic function integration setup	$u_{\min} = 10^{-6}$, $u_{\max} = 200$, $n_u = 1200$
Shared conventions	Put option, strike $K = 1$, THETA sign convention (THETA $= -\partial V/\partial\tau$), MAPE floor 10^{-4}
Arithmetic context	Double precision on CPU

Source: Own elaboration

5.5.2. Ablation Metrics By Greek

Greek quality is reported with the same primary metrics used in the rest of the chapter: MSE, RMSE, and MAPE (%).

TABLE 5.8. GREEK METRICS FOR BASELINE AND SOBOLEV PINN ($N = 2.5921 \times 10^4$ PER GREEK).

Variant	Greek	MSE	RMSE	MAPE (%)
Baseline	DELTA	1.11×10^{-3}	3.33×10^{-2}	10.74
Baseline	GAMMA	1.48×10^{-1}	3.85×10^{-1}	82.98
Baseline	VEGA	4.30×10^{-3}	6.55×10^{-2}	71.84
Baseline	THETA	2.32×10^{-5}	4.82×10^{-3}	31.98
Baseline	RHO	7.01×10^{-3}	8.37×10^{-2}	95.00
Sobolev	DELTA	1.23×10^{-7}	3.51×10^{-4}	0.28
Sobolev	GAMMA	7.99×10^{-5}	8.94×10^{-3}	31.29
Sobolev	VEGA	1.00×10^{-5}	3.17×10^{-3}	36.11
Sobolev	THETA	2.81×10^{-7}	5.30×10^{-4}	1.88
Sobolev	RHO	2.05×10^{-6}	1.43×10^{-3}	9.05

Source: Own elaboration

Compared with the baseline PINN, Sobolev augmentation reduces RMSE by approximately 9× (THETA) to 95× (DELTA), with the largest practical gains in the previously hardest Greeks (GAMMA and VEGA). Greek-inference throughput decreases from 9.48×10^3 to 8.60×10^3 points/s (~ 9.3% reduction), which remains a clearly favorable trade-off given the global accuracy gains.

5.5.3. Visual Diagnostics

This subsection is organized as a before/after narrative. Baseline diagnostics are first used to localize the dominant Greek-error modes, and Sobolev diagnostics are then read with the same visual layout to make improvements directly comparable.

Figure 5.4 displays baseline relative absolute-error maps for the five Greeks. The spatial pattern is heterogeneous, with broad high-error regions in GAMMA and VEGA, which is consistent with their weaker baseline MSE/RMSE values in Table 5.8.

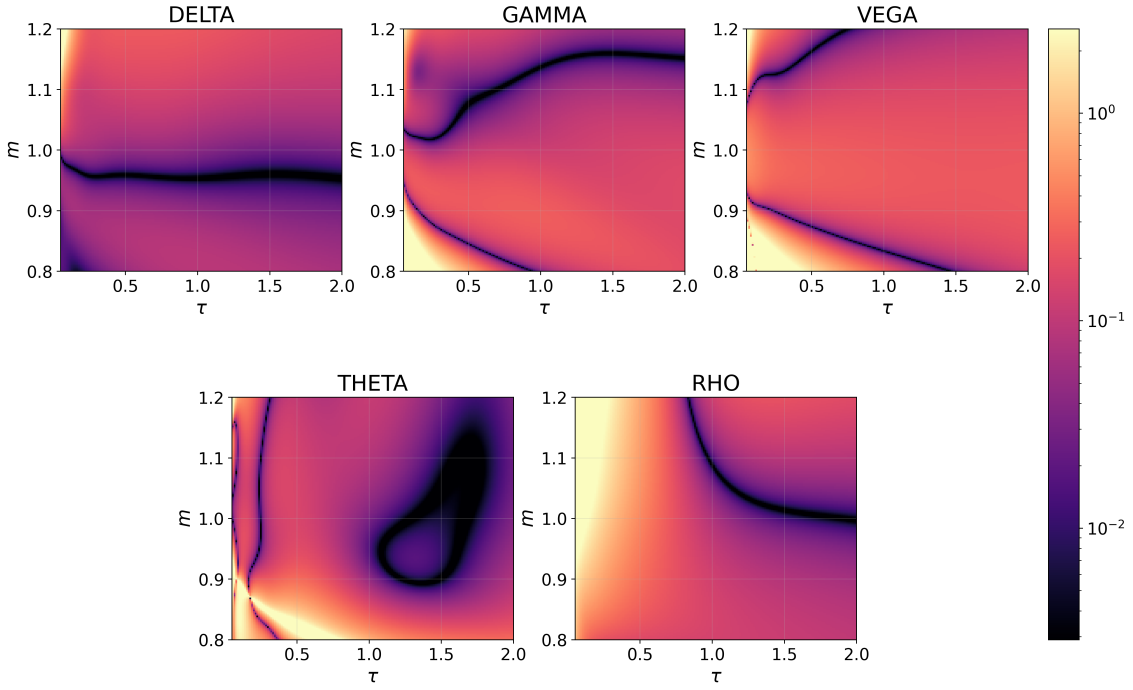


Fig. 5.3. Baseline PINN Greek relative-error maps.

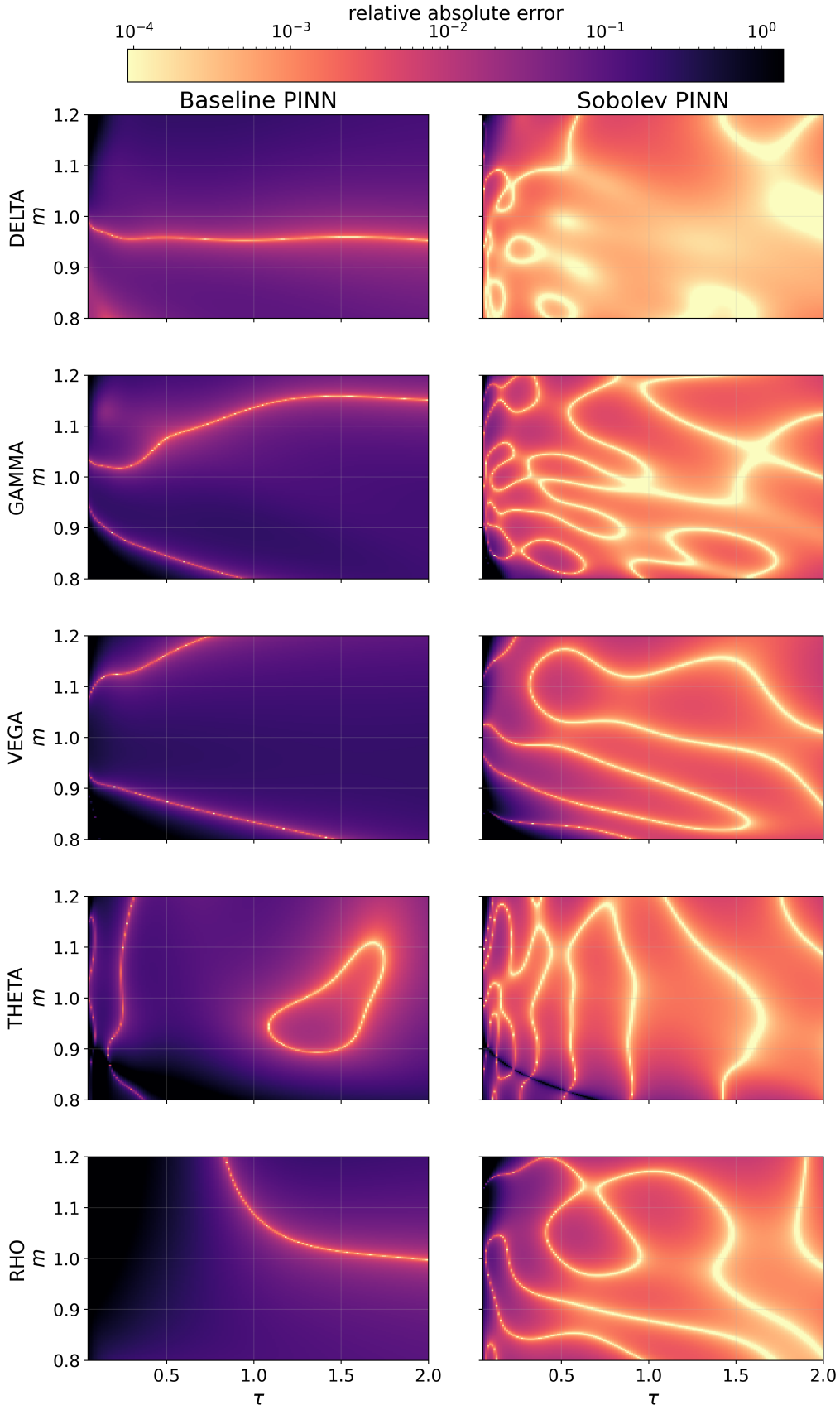


Fig. 5.4. Baseline PINN Greek relative-error maps.

The baseline histogram panel complements this map-level view with error distributions. The heaviest tails appear in GAMMA and VEGA, while DELTA, THETA, and RHO are comparatively tighter. This confirms that the baseline model captures coarse sensitivity structure but still exhibits non-negligible extreme deviations in the hardest derivatives.

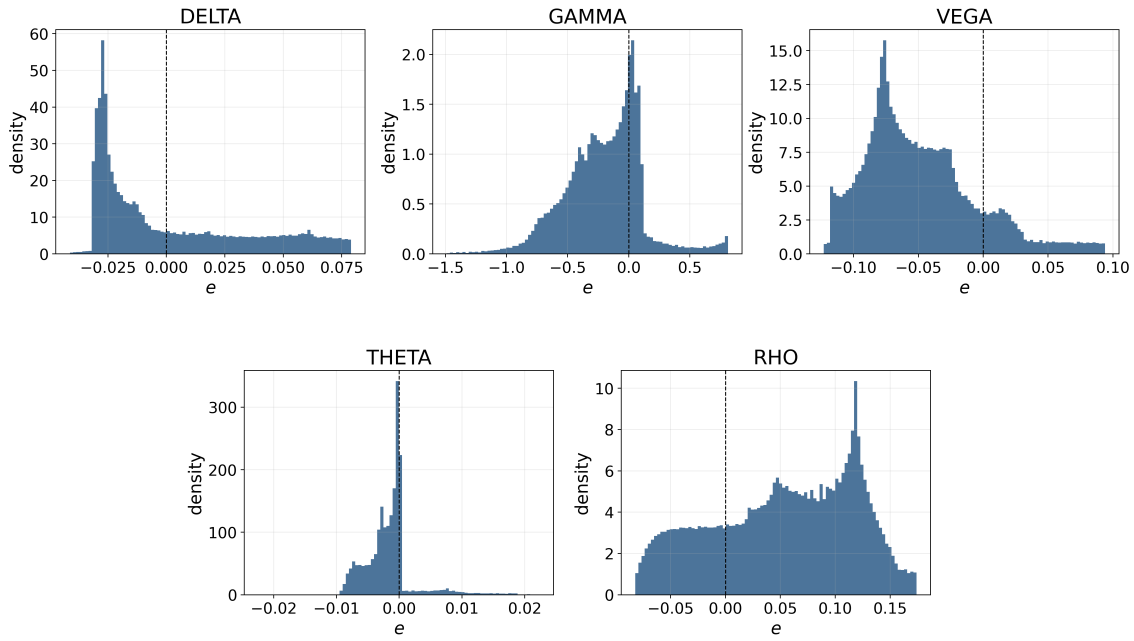


Fig. 5.5. Baseline PINN Greek error histograms.

After adding Sobolev terms, Figure 5.6 shows a clear contraction of high-error regions across all Greeks. The strongest visual improvement appears in GAMMA and VEGA, where problematic zones become much smaller and less intense than in the baseline maps. This spatial change is consistent with the order-of-magnitude RMSE reductions reported in Table 5.8.

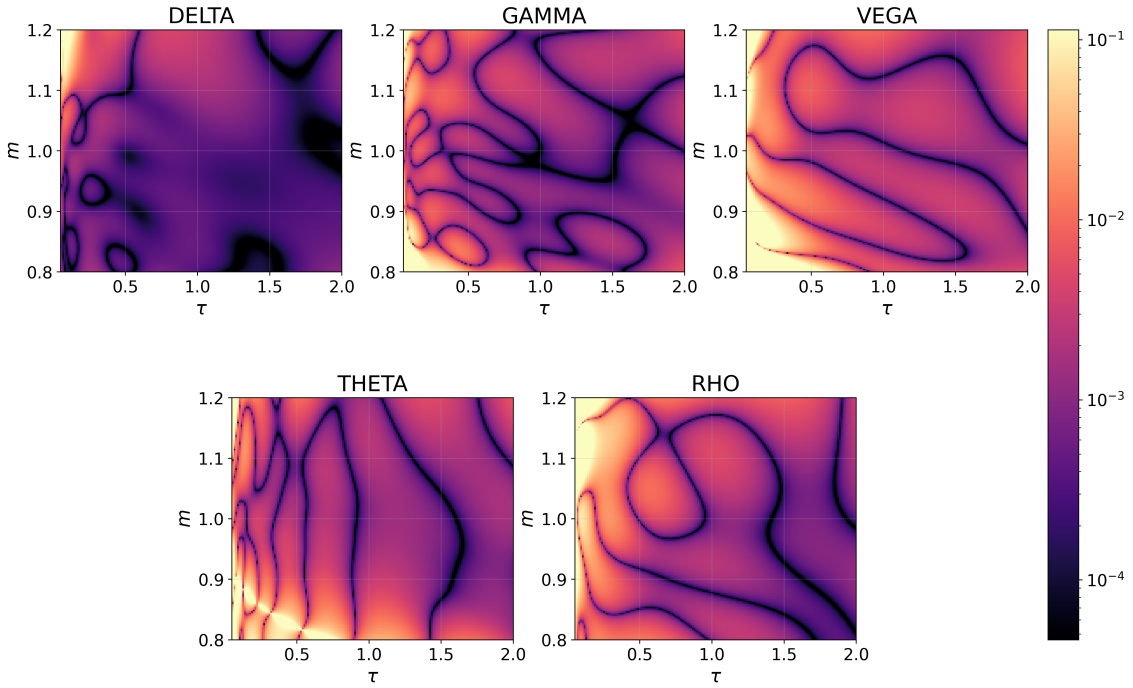


Fig. 5.6. Sobolev PINN Greek relative-error maps.

The same effect appears in Figure 5.7: Sobolev training narrows the distributions and reduces tail mass, so large deviations become less frequent for all five Greeks compared with the baseline histogram panel. The tail compression is particularly visible in GAMMA and VEGA, where baseline outliers were most persistent.

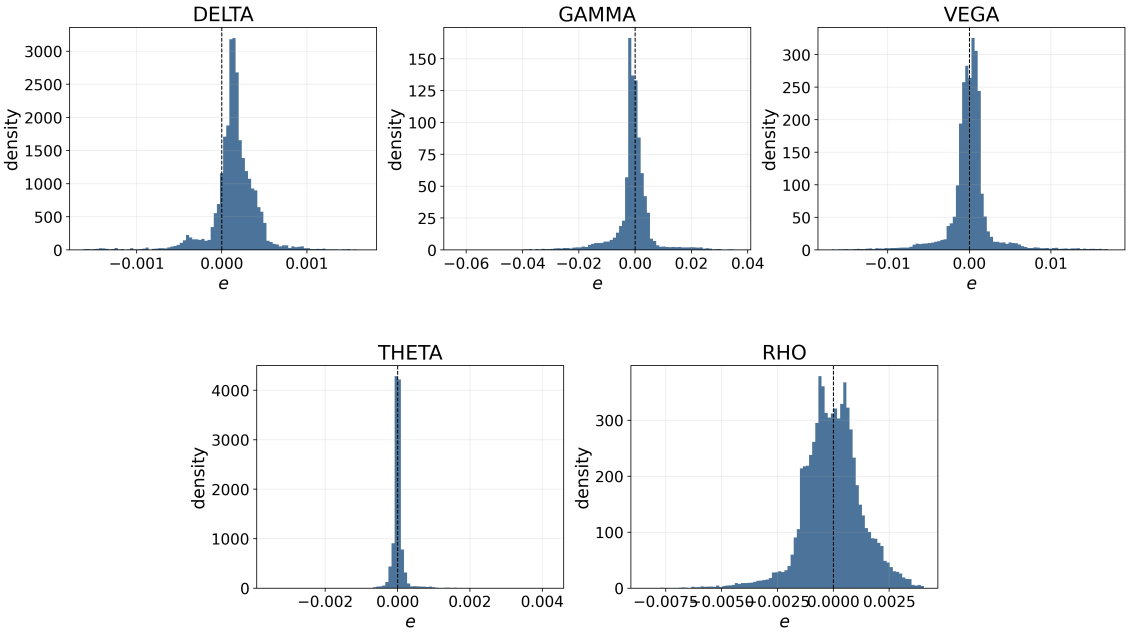


Fig. 5.7. Sobolev PINN Greek error histograms.

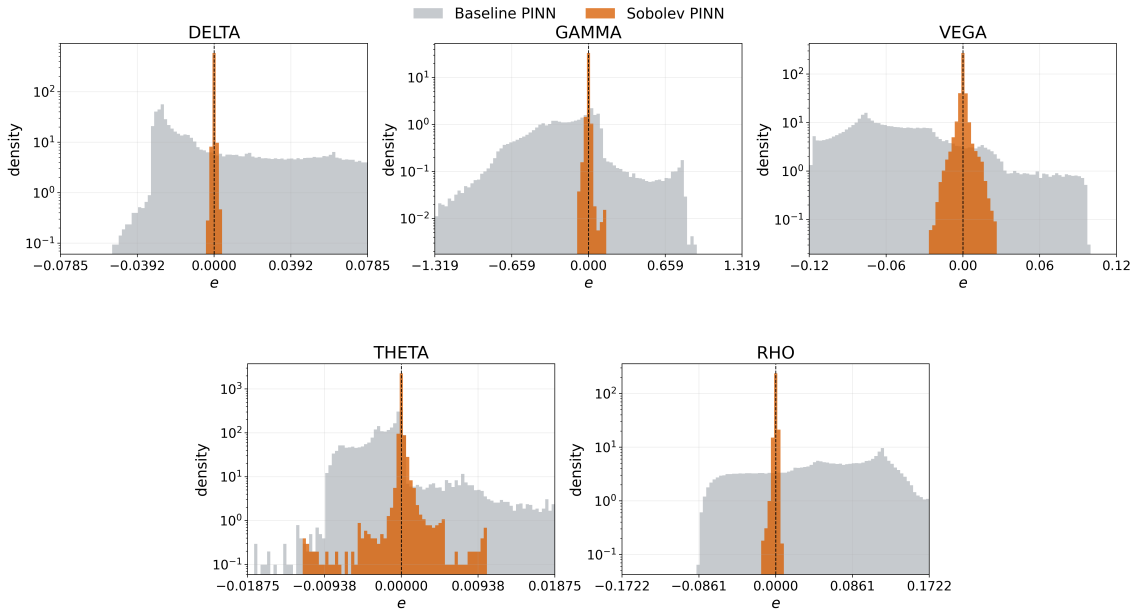


Fig. 5.8. **TODO: REUBICAR / AJUSTAR CAPTION.** Baseline–Sobolev Greek error histograms with overlaid log-density scale.

The two-phase optimization schedule used in this ablation trains epochs 0 to 4000 without Sobolev terms, then activates Sobolev augmentation from epoch 4000 onward. Together, the diagnostics indicate that Sobolev augmentation improves local derivative fidelity, not just aggregate scalar scores, while keeping the runtime overhead limited.

5.5.4. Control-Variate Greek Diagnostic

An additional Greek-focused diagnostic was run in parallel with the Sobolev study using an analytic-control-variate (ACV) hard patch. The accepted best-gate candidate keeps the frozen baseline PINN, adds a local Black–Scholes put control variate around the short-maturity ATM layer [2], and initializes the residual network at zero without residual training. Consequently, the result isolates the effect of the structural control variate rather than an extra supervised Greek fit.

In this diagnostic, "hard" refers to the short-maturity ATM region

$$\mathcal{D}_{\text{hard}} = \{(m, \tau) : |\log(m)| < 0.03,\ \tau < 0.05\},$$

which is precisely the neighborhood where the terminal put payoff kink at $m = 1, \tau = 0$ creates the strongest curvature stress for Gamma estimation.

TABLE 5.9. ACV CONTROL-VARIATE DIAGNOSTIC AGAINST THE ABLATION BASELINE.

Metric	Baseline PINN	ACV control variate
Price RMSE	4.269×10^{-3}	4.235×10^{-3}
Delta RMSE	3.381×10^{-2}	3.269×10^{-2}
Gamma RMSE	5.348×10^{-1}	4.040×10^{-1}
Hard Delta RMSE	8.729×10^{-2}	1.288×10^{-2}
Hard Gamma RMSE	4.992	6.446×10^{-1}
Hard p_{99} Gamma error	17.643	1.612

Source: Own elaboration based on ablation-study outputs.

The control variate is therefore the strongest candidate in the ablation table for full Delta RMSE, full Gamma RMSE, hard-region Delta RMSE, hard-region Gamma RMSE, and hard p_{99} Gamma error. The result should still be read cautiously near the terminal payoff kink: the remaining Gamma error is concentrated in the short-maturity ATM layer, and the extremely short bucket $\tau < 5 \times 10^{-4}$ still requires separate validation of the semi-analytical Gamma reference before making stronger claims there.

5.6. Cross-Model Computational Comparison

This section summarizes the end-to-end pipeline in operational order, emphasizing where each model contributes and how computational cost is distributed.

First, market calibration is driven by the ANN pricer plus the CaNN optimization loop. The ANN stage provides fast implied-volatility evaluations ($\approx 8.43 \times 10^5$ points/s), while CaNN concentrates cost in iterative inverse fitting. The selected tanh calibration reaches quote-level residual RMSE 2.7254×10^{-5} and improves all reported Liu et al. parameter errors on the replicated 35-quote benchmark.

Second, the calibrated market state $(\rho, \kappa, \gamma, \bar{v}, v_0)$ is injected into the parametric PINN pricing stage. In this configuration, baseline PINN pricing reaches MSE 3.16×10^{-5}, RMSE 5.62×10^{-3}, and MAPE 16.01%, while running at $\approx 4.91 \times 10^5$ points/s. Under the same benchmark protocol, COS runs at ≈ 87.36 points/s, so PINN remains several orders of magnitude faster in repeated evaluation mode.

Third, Greek extraction is performed on top of the same calibrated parameter state. The baseline PINN Greek pass runs at $\approx 9.48 \times 10^3$ points/s, while the Sobolev variant runs at $\approx 8.60 \times 10^3$ points/s (about 9.3% slower) and substantially improves derivative accuracy (for example, GAMMA RMSE from 3.85×10^{-1} to 8.94×10^{-3}, VEGA RMSE from 6.55×10^{-2} to 3.17×10^{-3}). In the separate hard-region ablation, the ACV control variate gives the best Delta/Gamma metrics, reducing hard Gamma RMSE from 4.992 to

6.446×10^{-1} and hard p_{99} Gamma error from 17.643 to 1.612.

Pipeline-level error should be read as stagewise bounds, not as a single scalar. The ANN and CaNN stages control IV-space mismatch used for calibration, pricing quality is validated in price space against COS, and Greek quality is validated in derivative space. Since these are different output spaces and objectives, the most reliable reading is the per-stage error envelope reported above and in Sections 5.2–5.5.

6. CONCLUSION AND FUTURE WORK

6.1. Main Conclusions

This thesis has developed and benchmarked an end-to-end computational workflow for Heston-based derivative analytics. The workflow starts from controlled data generation and numerical reference pricing, builds a supervised ANN implied-volatility surrogate, uses that surrogate inside a neural calibration loop, and then evaluates PINN-based pricing and Greek computation against semi-analytical references. The main outcome is not a single model, but an executable and auditable pipeline in which each stage can be tested independently and then connected to the next one.

The first conclusion is that the supervised ANN surrogate remains a useful acceleration layer for calibration. The ANN benchmark shows that the implied-volatility map can be approximated with errors in the 10^{-3} IV range on held-out data, while preserving very fast inference. This makes it suitable as a forward evaluator inside repeated inverse-problem calls, provided that the input remains inside the interpolation region and that boundary/extrapolation zones are treated with caution.

The strongest calibration result is obtained with the Liu-like tanh ANN used in the final CaNN benchmark. On the replicated 35-quote Liu grid [6], the selected run reaches weighted MSE 7.43×10^{-10} and residual RMSE 2.73×10^{-5} in IV units. More importantly, all reported Heston parameter errors improve relative to the Liu et al. reference values [6] under the same synthetic-truth comparison reported in Chapter 5. This is the clearest positive empirical result of the thesis: the calibration component does not merely reproduce the reference neural-calibration framework, but improves its parameter recovery in the benchmarked setting.

The second conclusion is that PINNs are effective as fast differentiable pricing surfaces, but their accuracy profile is more delicate than the ANN calibration component. The baseline PINN pricing benchmark reaches RMSE 5.62×10^{-3} against the COS reference [9] while giving a throughput several thousand times higher than the reference COS loop in repeated evaluation mode. However, the spatial diagnostics show that relative errors concentrate in low-price regions and near short maturities. This confirms that the PINN should be read as a fast differentiable approximation with identifiable failure modes, not as a universal replacement for the numerical solver.

The third conclusion concerns Greeks. Automatic differentiation makes Greeks available directly from the learned price surface, but accurate prices alone are not enough to guarantee accurate derivatives, especially for curvature quantities such as Gamma. Two strategies improved this part of the pipeline. Sobolev augmentation [16] substantially reduced global Greek errors by adding derivative information during training. In paral-

lel, the ACV control-variate diagnostic gave the strongest hard-region Delta and Gamma results without adding supervised Greek training to the residual patch. In the hard short-maturity ATM region, Gamma RMSE was reduced from 4.992 to 6.446×10^{-1}, and the hard p_{99} Gamma error from 17.643 to 1.612.

Taken together, these results support a practical division of roles. The ANN/CaNN block is the most mature part of the workflow and gives a strong calibration result. The PINN block is valuable as a differentiable pricing and sensitivity engine, but its derivative quality requires additional structure, either through Sobolev-style derivative supervision or through analytic control variates tailored to the singular region.

6.2. Contributions

The first contribution is a reproducible benchmark pipeline. All stages of the study are organized around explicit configurations, fixed data splits, numerical reference settings, and stored diagnostics. This matters because model comparisons in derivative pricing can be misleading if solver tolerances, data filters, or evaluation grids change silently between experiments.

The second contribution is the improved neural calibration result. The final tanh CaNN run preserves the basic Liu-style calibration idea [6], but obtains a stronger inverse fit on the benchmark quote set. The improvement appears both in quote-level IV residuals and in parameter recovery, which is the relevant quantity for a calibration workflow.

The third contribution is the PINN Greek analysis. The thesis does not stop at price RMSE; it evaluates first- and second-order sensitivities against semi-analytical Heston references and identifies where derivative errors concentrate. This is important because a pricing surrogate that is visually accurate can still be unusable for risk management if its local slopes and curvatures are unstable.

The fourth contribution is the hard-region ablation. The experiments show that several seemingly natural modifications, such as payoff-aware parametrizations or adaptive collocation, can improve one metric while degrading another. The accepted ACV control-variate result is therefore not just another model variant: it is the outcome of a targeted ablation process focused on the region where the payoff kink makes Gamma hardest to approximate.

6.3. Limitations

The main limitation is that the benchmark remains synthetic and Heston-specific. This is appropriate for controlled methodological evaluation, but it does not prove robustness on noisy market surfaces, sparse quotes, bid-ask spreads, dividends, or changing interest-rate conventions. The calibration result should therefore be read as a strong controlled

benchmark, not as a final production calibration study.

A second limitation is the dependence on vanilla European structure. The COS reference, the semi-analytical Greek engine, and the ACV correction are all built around a setting where strong analytical information is available. This is useful for validation, but portability to exotic derivatives or path-dependent contracts would require replacing parts of the reference and diagnostic stack.

A third limitation is the payoff kink itself. The ACV result reduces the short-maturity ATM Greek error substantially, but it does not remove the mathematical difficulty created by the non-smooth terminal payoff. In particular, the extremely short-maturity bucket $\tau < 5 \times 10^{-4}$ still requires careful validation of the reference Gamma before making stronger claims about that layer.

Finally, the thesis evaluates computational speed as a research benchmark rather than as a production latency target. The code is executable and scalable in structure, but it has not been optimized as a low-latency service or validated under portfolio-scale operational constraints.

6.4. Future Work

The first future direction is market-data validation. The CaNN result should be tested on real implied-volatility surfaces with realistic quote filtering, liquidity weights, and bid-ask uncertainty. This would show whether the improved synthetic calibration transfers to market conditions and whether the parameter recovery remains stable when the truth is no longer known.

The second direction is a hedging-oriented evaluation of Greeks. The present thesis verifies Greeks pointwise against reference values, which is the necessary first step. A natural continuation is to connect the Greek engine to a simple hedging backtest or risk-reporting workflow, measuring whether improved Delta and Gamma translate into more stable hedge rebalancing and P&L attribution.

The third direction is model and payoff extension. The executable structure of the pipeline makes it possible to test alternative stochastic-volatility models, local-volatility variants, or other payoff classes. For exotic derivatives, the main challenge will be to replace vanilla-specific analytical references with suitable numerical or simulation-based validation tools.

The fourth direction is further machine-learning improvement. Promising candidates include better loss balancing for PINNs, residual-gradient penalties in the spirit of gPINNs [11], adaptive sampling with stricter price-preservation criteria, and architectures designed specifically for stable second derivatives. The ablation results in this thesis suggest that such improvements should be accepted only if they improve hard-region Greek accuracy without degrading global pricing quality.

Overall, the thesis demonstrates that neural methods can be useful in derivative analytics when they are embedded in a disciplined numerical benchmark. The strongest evidence is the improved CaNN calibration result and the hard-region Greek improvement obtained with the ACV control variate. The remaining work is to move from controlled Heston benchmarks to noisier market and portfolio settings while preserving the same level of diagnostic discipline.

BIBLIOGRAPHY

[1] J. C. Hull, *Options, Futures, and Other Derivatives*, 11th ed. Pearson, 2022.

[2] F. Black and M. Scholes, "The pricing of options and corporate liabilities," *Journal of Political Economy*, vol. 81, no. 3, pp. 637–654, 1973. DOI: 10.1086/260062.

[3] S. L. Heston, "A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options," en, *The Review of Financial Studies*, vol. 6, no. 2, pp. 327–343, 1993. DOI: 10.1093/rfs/6.2.327.

[4] P. P. Boyle, "Options: A Monte Carlo approach," *Journal of Financial Economics*, vol. 4, no. 3, pp. 323–338, 1977. DOI: 10.1016/0304-405X(77)90005-8.

[5] P. Glasserman, *Monte Carlo Methods in Financial Engineering* (Stochastic Modelling and Applied Probability). Springer, 2004, vol. 53. DOI: 10.1007/978-0-387-21617-1.

[6] S. Liu, A. Borovykh, L. A. Grzelak, and C. W. Oosterlee, "A neural network-based framework for financial model calibration," en, *Journal of Mathematics in Industry*, vol. 9, no. 1, p. 9, Dec. 2019. DOI: 10.1186/s13362-019-0066-7. Accessed: Oct. 21, 2025. [Online]. Available: https://mathematicsinindustry.springeropen.com/articles/10.1186/s13362-019-0066-7.

[7] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, "Automatic Differentiation in Machine Learning: A Survey," *Journal of Machine Learning Research*, vol. 18, no. 153, pp. 1–43, 2018. [Online]. Available: http://jmlr.org/papers/v18/17-468.html.

[8] P. Carr and D. B. Madan, "Option valuation using the fast fourier transform," *Journal of Computational Finance*, vol. 2, no. 4, pp. 61–73, 1999. DOI: 10.21314/JCF.1999.043.

[9] F. Fang and C. W. Oosterlee, "A novel pricing method for european options based on fourier-cosine series expansions," *SIAM Journal on Scientific Computing*, vol. 31, no. 2, pp. 826–848, 2009. DOI: 10.1137/080718061.

[10] D. Hainaut and A. Casas, "Option pricing in the heston model with physics inspired neural networks," en, *Annals of Finance*, vol. 20, no. 3, pp. 353–376, 2024. DOI: 10.1007/s10436-024-00452-7. [Online]. Available: https://doi.org/10.1007/s10436-024-00452-7.

[11] J. Yu, L. Lu, X. Meng, and G. E. Karniadakis, "Gradient-enhanced physics-informed neural networks for forward and inverse PDE problems," *Computer Methods in Applied Mechanics and Engineering*, vol. 393, p. 114 823, 2022. DOI: 10.1016/j.cma.2022.114823. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0045782522001438.

[12] K. A. Malandain, S. Kalici, and H. Chakhoyan, *DeepSVM: Learning stochastic volatility models with physics-informed deep operator networks*, 2025. DOI: 10.48550/arXiv.2512.07162. arXiv: 2512.07162 [q-fin.CP]. [Online]. Available: https://arxiv.org/abs/2512.07162.

[13] S. Wang, Y. Teng, and P. Perdikaris, "Understanding and mitigating gradient flow pathologies in physics-informed neural networks," *SIAM Journal on Scientific Computing*, vol. 43, no. 5, A3055–A3081, 2021.

[14] S. Wang, X. Yu, and P. Perdikaris, "When and why PINNs fail to train: A neural tangent kernel perspective," *Journal of Computational Physics*, vol. 449, p. 110 768, 2022. DOI: 10.1016/j.jcp.2021.110768. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S002199912100663X.

[15] J. F. Urbán, P. Stefanou, and J. A. Pons, "Unveiling the optimization process of physics informed neural networks: How accurate and competitive can PINNs be?" en, *Journal of Computational Physics*, vol. 523, p. 113 656, Feb. 2025. DOI: 10.1016/j.jcp.2024.113656. Accessed: Mar. 19, 2026. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0021999124009045.

[16] W. M. Czarnecki, S. Osindero, M. Jaderberg, G. Swirszcz, and R. Pascanu, "Sobolev Training for Neural Networks," in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017. [Online]. Available: https://papers.nips.cc/paper/7015-sobolev-training-for-neural-networks.

[17] G. Amici, M. Morandotti, and C. Zhang, *Calibrating the heston model with deep differential networks*, 2024. DOI: 10.48550/arXiv.2407.15536. arXiv: 2407.15536 [q-fin.CP]. [Online]. Available: https://arxiv.org/abs/2407.15536.

[18] M. Noguer i Alonso, *Derivative-informed operator learning for finance: On-the-fly greeks, surfaces, hedging, and control*, 2026. DOI: 10.48550/arXiv.2606.05900. arXiv: 2606.05900 [q-fin.MF]. [Online]. Available: https://arxiv.org/abs/2606.05900.

[19] B. Oksendal, *Stochastic Differential Equations: An Introduction with Applications*, 6th ed. Springer, 2003.

[20] F. D. Rouah and S. L. Heston, *The Heston Model and its Extensions in Matlab and C#*. John Wiley & Sons, 2013.

[21] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3rd ed. Cambridge University Press, 2007.

[22] K. Levenberg, "A method for the solution of certain non-linear problems in least squares," *Quarterly of Applied Mathematics*, vol. 2, no. 2, pp. 164–168, 1944. DOI: 10.1090/qam/10666.

[23] D. W. Marquardt, "An algorithm for least-squares estimation of nonlinear parameters," *Journal of the Society for Industrial and Applied Mathematics*, vol. 11, no. 2, pp. 431–441, 1963. DOI: 10.1137/0111030.

[24] M. D. Mckay, R. J. Beckman, and W. J. Conover, "A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output From a Computer Code," en, *Technometrics*, vol. 42, no. 1, pp. 55–61, Feb. 2000. DOI: 10.1080/00401706.2000.10485979. Accessed: Mar. 21, 2026. [Online]. Available: http://www.tandfonline.com/doi/abs/10.1080/00401706.2000.10485979.

[25] G. Cybenko, "Approximation by superpositions of a sigmoidal function," *Mathematics of Control, Signals and Systems*, vol. 2, no. 4, pp. 303–314, 1989. DOI: 10.1007/BF02551274.

[26] K. Hornik, "Approximation capabilities of multilayer feedforward networks," *Neural Networks*, vol. 4, no. 2, pp. 251–257, 1991. DOI: 10.1016/0893-6080(91)90009-T.

[27] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed. Springer, 2006.

A. ANALYSIS OF THE COS METHOD ERROR

This appendix complements the Fourier-COS method introduced in Section 2.5. Its purpose is only to make explicit how the COS reference error is controlled in the implementation, since the detailed pricing formula is already given in Chapter 2.

A.1. COS Error Control

Starting from the pricing integral in Equation (2.23), the COS method replaces the transition density by the truncated cosine expansion in Equations (2.21)–(2.22). Following Fang and Oosterlee [9], the numerical error can be read as the sum of three effects: truncating the integration domain to $[a, b]$, using only N cosine terms, and approximating the exact density coefficients by characteristic-function-based coefficients. In the shorter notation used in Equation (2.27),

$$|V - V_N^{\mathrm{COS}}| \leq \varepsilon_{\mathrm{trunc}}([a, b]) + \varepsilon_N.$$

For plain-vanilla European options, the payoff coefficients H_k in Equation (2.25) are available in closed form, so no separate payoff quadrature error is introduced.

The truncation interval is selected from cumulants of the log-price state. Using the cumulants obtained from Equation (2.9), the implementation follows the standard cumulant rule

$$[a, b] = \left[c_1 - L\sqrt{c_2 + \sqrt{c_4}},\ c_1 + L\sqrt{c_2 + \sqrt{c_4}} \right],$$

which is the interval stated in Equation (2.26). Increasing L reduces probability-mass truncation error, but it also widens the interval and may require a larger N to preserve resolution. Therefore, L and N are treated jointly rather than as independent numerical knobs.

A.2. Reference Configuration

In the experiments, the reference configuration uses $N = 1500$ and $L = 50$, with fallback $L = 80$ in low-IV or otherwise unstable inversion cases. These settings are deliberately conservative because COS prices are used downstream as ANN labels, implied-volatility inversion inputs, and PINN benchmark references. The benchmark protocol in Section 5.1 therefore checks COS convergence with respect to (N, L), put-call parity, and Black–Scholes consistency in near-constant-variance regimes before neural-model errors are interpreted.

B. CALIBRATION OF THE HESTON MODEL

TODO:

C. PINN PRICING BENCHMARK SETUP

The pricing benchmark compares a baseline parametric PINN against the COS/Heston reference under a common evaluation protocol and shared numerical conventions.

The evaluated model is the baseline parametric PINN without Sobolev terms. Inputs follow the 8-dimensional convention

$$[\tau, m, v, \rho, \kappa, \gamma, \bar{v}, r],$$

with affine standardization fitted on training statistics. Concretely, each feature x_j is transformed as

$$\tilde{x}_j = \frac{x_j - \mu_j^{\text{train}}}{\sigma_j^{\text{train}}},$$

where μ_j^{train} and σ_j^{train} are computed only on the training split and then reused unchanged in validation, test, and benchmark evaluation. This keeps features on comparable scales and stabilizes training and gradient computation, which is especially important in PINN settings; it also avoids data leakage from evaluation sets.

The evaluation sample contains 1.0×10^4 candidates, with 9.996×10^3 valid comparisons after numerical filtering. COS pricing is computed with $N = 1500$, $L = 50$, and fallback $L = 80$, under European put convention ($K = 1$) and no-arbitrage tolerance 10^{-8}. All reported metrics are computed in double precision on CPU.

D. GLOBAL NOTATION GLOSSARY

This appendix contains a global glossary of symbols used throughout the thesis. It is intended as a living reference and can be expanded as additional notation appears in later revisions.

TABLE D.1. GLOBAL NOTATION GLOSSARY: CORE PRICING AND MODEL NOTATION

Symbol	Meaning
$(\Omega, \mathcal{F}, (\mathcal{F}_t), \mathbb{P})$	Filtered probability space: outcomes, measurable events, information available over time, and original probability measure.
$t,\ T,\ \tau$	Valuation time, maturity, and time to maturity ($\tau = T - t$).
$S_t,\ K,\ m$	Spot price, strike, and moneyness ($m = S_0/K$ for quote grids, $m = S/K$ as a PDE coordinate).
r	Constant risk-free interest rate.
$\mathbb{Q}$	Risk-neutral probability measure.
V_t	Arbitrage-free option value at time t.
$V(t_0, x)$	Option value at valuation time t_0 when the state variable is x.
$V_{BS},\ V_{\text{target}}$	Black–Scholes option value and target option value used in IV inversion.
$\Psi,\ \Psi_{\text{put}}$	Option payoff function at maturity; for the thesis pricing target, $\Psi_{\text{put}} = (K - S_T)^+$.
$X_t,\ x,\ y$	State variable and its initial/terminal values, typically $x = X(t_0)$ and $y = X(T)$.
$F_X,\ f_X$	Cumulative distribution function and probability density function of X.
θ	Generic model-parameter vector.
μ	Generic drift under the original probability measure.
$W_t^{(S)},\ W_t^{(v)}$	Brownian motions driving the spot and variance processes.
$\kappa,\ \bar{\nu},\ \gamma,\ \rho,\ \nu_0$	Heston parameters: mean reversion, long-run variance, vol-of-vol, correlation, and initial variance. Later chapters also use $\bar{v}$ and v_0 for the variance parameters.

Source: Author's elaboration

TABLE D.2. GLOBAL NOTATION GLOSSARY: TRANSFORM, COS, AND IV NOTATION

Symbol	Meaning
$\phi_X(u)$	Characteristic function of random variable X.
$\mathcal{M}_X(u),\ \zeta_X(u),\ \zeta_k$	Moment generating function, cumulant characteristic function, and k-th cumulant.
$u,\ u_k,\ i$	Fourier variable, COS frequency $u_k = k\pi/(b-a)$, and imaginary unit.
$[a,b],\ L,\ N,\ k$	COS truncation interval, truncation scaling parameter, number of terms, and series index.
$\bar{F}_k,\ H_k,\ U_k$	COS density, payoff, and Heston-COS payoff coefficients.
$\hat{V}_T(u),\ V_N^{\mathrm{COS}}$	Fourier transform of the maturity payoff and N-term COS price approximation.
$\varphi_H,\ C,\ D,\ d,\ g$	Heston characteristic function and its affine auxiliary functions.
$\chi_k,\ \psi_k$	Closed-form payoff integrals used in COS coefficients.
σ_{imp}	Implied volatility obtained from numerical inversion.
$F(\sigma),\ \varepsilon_{\mathrm{price}},\ \varepsilon_\sigma,\ n_{\max}$	IV root function, price tolerance, volatility tolerance, and maximum number of solver iterations.
$\hat{z}_i,\ z_i^{\mathrm{ref}}$	Model and reference values for a generic benchmark target at evaluation point i.
MSE, RMSE, MAPE	Mean squared error, root mean squared error, and mean absolute percentage error used for benchmark reporting.
$\Pi_j,\ \Psi_j$	Heston call-pricing probability terms and their Fourier integrands.
Δ, Γ, Vega, Rho, Θ	Standard option Greeks: spot first derivative, spot second derivative, variance/volatility sensitivity, interest-rate sensitivity, and time sensitivity.

Source: Author's elaboration